

الجمهورية العربية السورية
المعهد العالي للعلوم التطبيقية والتكنولوجيا
قسم المعلومات
العام الدراسي 2024/2025

مشروع السنة الرابعة

تطبيق مساعد للتمارين الرياضية

تقديم

سعد هلال أسود

إشراف

د. رياض سنبل

م. مرج حسن

2025/1/8

إهداء

حكمة

كلمة شكر

الخلاصة

في السنوات الأخيرة، كان هناك تحول كبير نحو الصحة والعافية، مع تزايد الطلب على حلول لياقة بدنية سهلة الوصول وشخصية وجذابة. يواجه العديد من الأفراد صعوبة في العثور على منصة واحدة متماسكة يمكنها تلبية احتياجاتهم التدريبية المحددة، وتتبع تقدمهم بفعالية، وإبقائهم متحمسين. غالبًا ما تكون التطبيقات الحالية إما عامة جدًا أو معقدة جدًا أو تقتصر إلى البنية التحتية القوية لتوفير تجربة مستخدم موثوقة. تم تصميم هذا المشروع، لسد هذه الفجوة. كانت الفكرة الأساسية هي إنشاء تطبيق لياقة بدنية متكامل لا يوفر للمستخدمين الأدوات اللازمة لإدارة صحتهم فحسب، بل يوفر أيضًا واجهة خلفية قابلة للتطوير وعالية الأداء لضمان تجربة سلسة. من خلال الجمع بين تطبيق محمول متعدد المنصات للمستخدمين ولوحة تحكم إدارية قوية لإدارة المحتوى، يهدف VexaFit إلى أن يكون حلاً شاملاً لعشاق اللياقة البدنية العصريين.

Abstract

In recent years, there has been a significant shift towards health and wellness, with a growing demand for accessible, personalized, and engaging fitness solutions. Many individuals struggle to find a single, cohesive platform that can cater to their specific workout needs, track their progress effectively, and keep them motivated. Existing applications are often either too generic, too complex, or lack the robust infrastructure to provide a reliable user experience. This project was conceived to address this gap. The core idea was to create a full-stack fitness application that not only provides users with the tools to manage their health but also offers a scalable and high-performance backend to ensure a seamless experience. By combining a cross-platform mobile app for users with a powerful admin dashboard for content management, VexaFit aims to be a comprehensive solution for the modern fitness enthusiast.

قائمة المحتويات

11	الفصل الأول: التعريف بالمشروع
11	1.1 مقدمة
11	2.1 أهداف المشروع
12	3.1 المتطلبات الوظيفية
12	1.3.1 متطلبات المستخدم
12	2.3.1 متطلبات المسؤول
13	4.1 المتطلبات غير الوظيفية
14	الفصل الثاني: الدراسة النظرية
14	1.2 المشهد المتطور للياقة البدنية الرقمية في المنزل
14	2.2 حجم السوق وتوقعات النمو
15	3.2 محركات السوق الرئيسية والعوامل التكنولوجية المساعدة
16	4.2 مكانة VexaFit الاستراتيجية
16	5.2 تحديات في اللياقة البدنية المنزلية الغير خاضعة للرقابة
17	6.2 الحواجز النفسية والتحفيزية
17	7.2 القيود البيئية واللوجستية
18	الفصل الثالث: الدراسة المرجعية
18	1.3 مقدمة
18	2.3 بعض التطبيقات الشائعة
18	1.2.3 رائد المحتوى المجاني (Nike Training Club)
19	2.2.3 تطبيق Peloton تجربة مجتمعية بقيادة المدربين
19	3.2.3 متتبع التغذية المعتمد على البيانات MyFitnessPal
20	3.3 خلاصة الدراسة المرجعية
21	الفصل الرابع: الدراسة التحليلية
21	1.4 مخططات حالات الاستخدام للمستخدم
24	2.4 السرد النصي لحالات الاستخدام للمستخدم
24	1.2.4 إنشاء حساب جديد
26	2.2.4 تسجيل الدخول
28	3.2.4 إنشاء قائمة تمارين خاصة بالمستخدم
30	4.2.4 إضافة تمرين إلى قائمة التمارين الخاصة بالمستخدم

32	5.2.4 عرض قوائم التمارين (المعدة مسبقاً والخاصة)
33	6.2.4 استعراض التمارين التي يدعمها التطبيق مصنفة حسب النوع
35	3.4 مخطط حالات الاستخدام للمسؤول
38	4.4 السرد النصي لحالات الاستخدام للمسؤول
38	1.4.4 عرض المستخدمين المسجلين وتغيير حالتهم
39	2.4.4 إضافة صنف جديد للتمارين
40	3.4.4 إضافة تمرين جديد
42	4.4.4 إنشاء قائمة تمارين معدة مسبقاً
43	الفصل الخامس: أسلوب العمل
43	1.5 مقدمة
43	2.5 نموذج التطوير المتبع
44	3.5 منهجية تحويل الدراسات إلى نظام
44	4.5 استخلاص المتطلبات من الأبحاث
44	5.5 نمذجة وظائف النظام
45	6.5 الفلسفة المعمارية: البنية التنظيمية (Clean Architecture)
45	7.5 الاختبارات
46	الفصل السادس: تصميم النظام
46	1.6 مقدمة
46	2.6 مخطط النظام التصميمي
47	3.6 البنية المعمارية للنظام (System Architecture)
47	4.6 بنية الواجهة الخلفية (.NET)
48	5.6 بنية الواجهة الأمامية (Flutter)
51	الفصل السابع: الأدوات المستخدمة
51	ASP.NET Core 1.7
51	Entity Framework Core 2.7
52	PostgreSQL 3.7
52	Json Web Token (JWT) 4.7
52	Flutter 5.7
52	React 6.7
53	xUnit 7.7
53	Moq 8.7

53	Grafana k6 9.7
54	Matplotlib 10.7
54	Git 11.7
54	Github 12.7
54	go_router 13.7
55	dio 14.7
55	provider 15.7
55	flutter_secure_storage 16.7
56	الفصل الثامن: تنفيذ النظام
56	1.8 مقدمة
56	2.8 تنفيذ الواجهة الخلفية (BackEnd)
57	3.8 تنفيذ الواجهة الأمامية (تطبيق الموبايل)
58	4.8 تنفيذ لوحة تحكم المستخدم
59	5.8 تصميم قاعدة المعطيات
60	6.8 مخطط النظام التنفيذي
61	الفصل التاسع: اختبارات النظام ومناقشة النتائج
61	1.9 اختبار خدمة إدارة عمليات النظام
61	1.1.9 التجهيز للاختبارات
62	2.1.9 اختبار الوحدات واختبارات التكامل واختبارات الحماية
65	3.1.9 اختبار الضغط
67	4.1.9 مناقشة النتائج
69	خاتمة وآفاق مستقبلية

قائمة الصور

21	حالات استخدام إدارة حسابات المستخدمين
22	حالات استخدام إدارة عمليات النظام للمستخدم
23	حالات استخدام إدارة عمليات النظام للمستخدم
26	مخطط SSD لحالة الاستخدام إنشاء حساب جديد
28	مخطط SSD لحالة الاستخدام تسجيل دخول المستخدم
30	مخطط SSD لحالة الاستخدام إنشاء قائمة تمارين خاصة بالمستخدم
32	مخطط SSD لحالة الاستخدام إضافة تمرين إلى قائمة التمارين الخاصة بالمستخدم
34	مخطط SSD لحالة الاستخدام استعراض التمارين
35	حالات استخدام إدارة حسابات المسؤول
36	حالات استخدام إدارة عمليات النظام للمسؤول
37	حالات استخدام إدارة عمليات النظام للمسؤول
46	مخطط النظام التصميمي
57	توضيح البنية لتجزير الواجهة الخلفية
57	توضيح البنية لتجزير الواجهة الأمامية للمستخدم
58	الصفحة الرئيسية لواجهة المسؤول
59	صفحة إدارة المستخدمين لواجهة المسؤول
59	تصميم قاعدة معطيات خدمة إدارة عمليات النظام
60	كيفية التواصل بين الخدمات ضمن النظام مع الأدوات المستخدمة
61	ملئ قاعدة البيانات بعلومات للاختبار
62	النتائج المحلية لاختبارات الوحدة ضمن خدمة إدارة العمليات
63	النتائج المحلية لاختبارات التكامل والحماية ضمن خدمة إدارة العمليات
64	تجميع الاستعلامات في استعلام واحد عن طريق Entity Framework
64	الخطأ الناتج عن استعمال الطريقة الموضحة في الصورة 17
64	تجميع الاستعلامات عن طريق Lambda Expression
65	السيناريوهات التي تم اعتمادها لإجراء اختبار الضغط
66	المحددات للزمن وعدد المستخدمين الافتراضيين لكل فترة ضمن كل سيناريو
67	نتائج اختبار الضغط
68	نتائج اختبار الضغط بعد تقسيم الاستعلام الخاص بقوائم التمارين
68	المقارنة بين الأزمنة لتخديم الطلبات قبل وبعد التعديل على طريقة استعلام قوائم التمارين حيث يظهر الخط الأزرق النتائج قبل التعديل

قائمة الجداول

15	توقعات سوق اللياقة البدنية المنزلية والتفاعلية العالمي(2033-2024)
20	مقارنة بين بعض التطبيقات المشابهة للتطبيق المقترح
24	السرد النصي لحالة استخدام إنشاء حساب جديد للمستخدم
24	السيناريو الناجح لحالة استخدام إنشاء حساب جديد للمستخدم
26	السرد النصي لحالة استخدام تسجيل الدخول للمستخدم
27	السيناريو الناجح لحالة استخدام تسجيل الدخول للمستخدم
28	السرد النصي لحالة استخدام إنشاء قائمة تمارين خاصة بالمستخدم
29	السيناريو الناجح لحالة استخدام إنشاء قائمة تمارين خاصة بالمستخدم
30	السرد النصي لحالة استخدام إضافة تمرين إلى قائمة التمارين الخاصة بالمستخدم
31	السيناريو الناجح لحالة استخدام إضافة تمرين إلى قائمة التمارين الخاصة بالمستخدم
32	السرد النصي لحالة استخدام عرض قوائم التمارين (المعدة مسبقاً والخاصة)
33	السيناريو الناجح لحالة استخدام عرض قوائم التمارين (المعدة مسبقاً والخاصة)
33	السرد النصي لحالة استخدام استعراض التمارين
34	السيناريو الناجح لحالة استخدام استعراض التمارين
38	السرد النصي لحالة استخدام عرض المستخدمين المسجلين وتغيير حالتهم
38	السيناريو الناجح لحالة استخدام عرض المستخدمين المسجلين وتغيير حالتهم
39	السرد النصي لحالة استخدام إضافة صنف جديد للتمارين
40	السيناريو الناجح لحالة استخدام إضافة صنف جديد للتمارين
40	السرد النصي لحالة استخدام إضافة تمرين جديد
41	السيناريو الناجح لحالة استخدام إضافة تمرين جديد
42	السرد النصي لحالة استخدام إنشاء قائمة تمارين معدة مسبقاً
42	السيناريو الناجح لحالة استخدام إنشاء قائمة تمارين معدة مسبقاً

الفصل الأول

التعريف بالمشروع

يتضمن هذا الفصل التعريف بالمشروع ومتطلباته.

1.1 مقدمة

يشهد العالم اليوم اهتماماً متزايداً بأنماط الحياة الصحية واللياقة البدنية، مما أدى إلى انتشار واسع لتطبيقات الهواتف المحمولة التي تهدف إلى مساعدة الأفراد في تحقيق أهدافهم الصحية. على الرغم من توفر العديد من هذه التطبيقات، إلا أن هناك فجوة واضحة في السوق تتمثل في غياب منصة متكاملة تجمع بين تجربة مستخدم سلسة، ومحتوى تدريبي مخصص، وبنية تحتية قوية وقابلة للتطوير. يعاني المستخدمون غالباً من تطبيقات إما أنها تقتصر إلى العمق الوظيفي أو تعاني من مشاكل في الأداء والموثوقية عند زيادة عدد المستخدمين.

من هذا المنطلق، يهدف المشروع إلى تصميم وتطوير نظام لياقة بدنية شامل يتكون من تطبيق محمول للمستخدمين النهائيين، مدعوماً بواجهة سهلة الاستخدام، بالإضافة إلى لوحة تحكم إدارية تتيح إدارة المحتوى والمستخدمين بكفاءة.

2.1 أهداف المشروع

يهدف هذا المشروع إلى تطوير تطبيق رياضي منزلي بسيط يمكن استخدامه من قبل الشباب والمهتمين بالحفاظ على صحتهم البدنية في أي وقت ومن أي مكان. يعمل التطبيق على توفير خطط تدريب مخصصة بحسب بيانات المستخدم مثل العمر والجنس والوزن، مع إمكانية تتبع التمارين اليومية وتسجيل الوجبات الغذائية. كما يسعى التطبيق إلى تحفيز المستخدمين من خلال إشعارات منتظمة، وسجل لتتبع التقدم والاستمرارية. يتميز التطبيق بسهولة الاستخدام، وواجهة تفاعلية واضحة، مما يجعله أداة عملية تساعد في بناء عادات صحية مستدامة دون تعقيد أو تكلفة.

3.1 المتطلبات الوظيفية

1.3.1 متطلبات المستخدم

1. إدارة الحسابات:
 - يجب أن يتمكن المستخدم من إنشاء حساب جديد في التطبيق باستخدام بريد إلكتروني وكلمة مرور.
 - يجب أن يتمكن المستخدم من تسجيل الدخول إلى حسابه بشكل آمن.
 - يجب أن يتمكن المستخدم من تعديل معلومات ملفه الشخصي وتغيير كلمة المرور.
2. إدارة قوائم التمارين:
 - يجب أن يتمكن المستخدم من إنشاء قوائم تمارين مخصصة (Custom Workouts) وتسميتها ووضع وصف لها.
 - يجب أن يتمكن المستخدم من إضافة تمارين من مكتبة التمارين إلى قوائمه الخاصة.
 - يجب أن يتمكن المستخدم من تعديل التمارين ضمن قوائمه (مثل تغيير عدد الجولات والتكرارات).
 - يجب أن يتمكن المستخدم من حذف التمارين من قوائمه أو حذف القوائم بأكملها.
3. استعراض المحتوى:
 - يجب أن يتمكن المستخدم من استعراض قوائم التمارين المعدة مسبقاً (Predefined Workouts) التي يوفرها النظام.
 - يجب أن يتمكن المستخدم من استعراض مكتبة التمارين الكاملة.
 - يجب أن يتمكن المستخدم من البحث عن تمارين محددة وتصنيفها بناءً على الفئات (مثل مجموعة العضلات أو نوع التمرين).
 - يجب أن يتمكن المستخدم من عرض تفاصيل كل تمرين، بما في ذلك الصور ومقاطع الفيديو التوضيحية.

2.3.1 متطلبات المسؤول

1. إدارة المستخدمين:
 - يجب أن يتمكن المسؤول من استعراض قائمة بجميع المستخدمين المسجلين في النظام.
 - يجب أن يتمكن المسؤول من تفعيل أو إلغاء تفعيل حسابات المستخدمين.
 - يجب أن يتمكن المسؤول من الاطلاع على إحصائيات عامة حول المستخدمين (مثل العدد الإجمالي، عدد المستخدمين النشطين).
2. إدارة المحتوى:
 - إدارة التمارين: يجب أن يتمكن المسؤول من إضافة تمارين جديدة إلى المكتبة، وتعديل تفاصيل التمارين الحالية (الاسم، الوصف، الصور، الفيديوها)، وحذف التمارين.

- إدارة الفئات: يجب أن يتمكن المسؤول من إنشاء فئات جديدة للتمارين (مثل "تمارين صدر" أو "تمارين كارديو")، وتعديلها، وحذفها.
- إدارة قوائم التمارين المعدة مسبقاً: يجب أن يتمكن المسؤول من إنشاء وتعديل وحذف قوائم التمارين التي تظهر لجميع المستخدمين.

4.1 المتطلبات غير الوظيفية

1. الأداء (Performance):
 - يجب ألا يتجاوز وقت استجابة واجهة برمجة التطبيقات 800 (API) مللي ثانية — 95% من الطلبات تحت ظروف الحمل العادية.
 - يجب أن يتم تحميل واجهات المستخدم في تطبيق الموبايل بسرعة وسلاسة لضمان تجربة مستخدم جيدة.
2. الموثوقية (Reliability):
 - يجب أن يكون معدل فشل الطلبات إلى الـ API أقل من 1% خلال التشغيل العادي.
 - يجب أن يكون النظام متاحاً للعمل بنسبة 99.5% من الوقت.
3. قابلية التوسع (Scalability):
 - يجب أن تكون بنية النظام قادرة على التعامل مع زيادة عدد المستخدمين والبيانات دون الحاجة إلى إعادة تصميم جذرية.
4. الأمان (Security):
 - يجب أن تتم جميع الاتصالات بين تطبيق الموبايل والواجهة الخلفية عبر بروتوكول HTTPS المشفر.
 - يجب تخزين كلمات مرور المستخدمين بشكل آمن باستخدام تقنيات التشفير (Hashing).
 - يجب أن تكون صلاحيات الوصول إلى وظائف لوحة التحكم مقتصرة على المستخدمين الذين لديهم دور "مسؤول" فقط.
5. قابلية الصيانة (Maintainability):
 - يجب أن يتم بناء النظام باستخدام معمارية نظيفة (Clean Architecture) لضمان أن يكون الكود منظماً وسهل الفهم والتعديل والإضافة إليه في المستقبل.

الفصل الثاني

الدراسة النظرية

يوضح هذا الفصل مجموعة من المفاهيم النظرية المستخدمة ضمن العمل.

1.2 المشهد المتطور للياقة البدنية الرقمية في المنزل

يشهد مجال الرياضة والصحة العالمية تحولاً جذرياً، يتميز بانقلاب كبير ومستمر من نماذج اللياقة البدنية التقليدية القائمة على وجود مدرب شخصي إلى الحلول الرقمية في المنزل. هذا التحول، الذي تسارع بفضل تطور أنماط حياة المستهلكين والتقدم التكنولوجي، قد أوجد بيئة اقتصادية وثقافية خصبة لتطبيقات مثل VexaFit. لتقدير سياق المشروع وإمكانياته بالكامل، من الضروري أولاً تأسيس فهم شامل لهذا التحول. سيقوم هذا القسم بتحليل حجم السوق ومسار نموه، وتشريح الدوافع الرئيسية والعوامل التكنولوجية التي تغذي توسعه^{[13][9][6]}.

2.2 حجم السوق وتوقعات النمو

قطاع اللياقة البدنية في المنزل ليس اتجاهًا عابراً بل سوقاً عالمياً قوياً ومتوسعاً. تؤكد تقارير تحليل السوق المتعددة هذا الأمر، وترسم صورة لنمو ثابت وكبير. في حين تختلف التقييمات المحددة بناءً على قطاعات السوق الدقيقة التي يتم تحليلها والتي تتراوح من المعدات المادية إلى المنصات التفاعلية الرقمية البحتة فإن الاستنتاج العام إيجابي بالإجماع.

وفقاً لـ Fortune Business Insights، يُقدَّر سوق معدات اللياقة البدنية المنزلية العالمي بـ 19.79 مليار دولار أمريكي بحلول عام 2032، بمعدل نمو سنوي مركب يبلغ 6.32%^[1]. وبالمثل، قدرت Straits Research قيمة السوق بـ 18.03 مليار دولار أمريكي بحلول عام 2033 بمعدل نمو سنوي أكثر قوة يبلغ 8.2%. وتتوقع تحليلات أخرى توسعاً مماثلاً، حيث تتوقع Market.us أن تصل قيمة السوق إلى 21.8 مليار دولار أمريكي بحلول عام 2033، نمواً من 13.4 مليار دولار أمريكي في عام 2023 بمعدل نمو سنوي مركب يبلغ 5.0%^{[9][6]}. وتقدم Technavio نظرة أكثر تفصيلاً، متوقعة نمواً قدره 4.44 مليار دولار أمريكي بين عامي 2024 و 2028، مدفوعاً بمعدل نمو سنوي مركب يبلغ 11.38%.

هذا التباين في توقعات النمو لا يدل على عدم اليقين، بل يعكس الطبيعة متعددة الأوجه لصناعة اللياقة البدنية المنزلية. غالباً ما تتعلق أرقام معدل النمو السنوي المركب المنخفضة بالسوق الأوسع الذي يشمل معدات تمارين القلب والقوة الثقيلة، في حين أن معدلات النمو الأعلى هي أكثر تميزاً للقطاعات المدمجة تكنولوجياً والتي تعتمد على البرمجيات. فُدرَّ سوق اللياقة البدنية التفاعلية، الذي يشمل مباشرة تطبيقات الهاتف المحمول بـ 5.73 مليار دولار أمريكي في عام 2024 ومن المتوقع أن يتوسع

بمعدل نمو سنوي مركب يبلغ 8.61%^[3]، ليصل إلى 13.09 مليار دولار أمريكي بحلول عام 2034^[6]. هذا التمييز حاسم، لأنه يضع الحلول التي تركز على البرمجيات ضمن القطاع الفرعي الأكثر ديناميكية وسرعة في التوسع في صناعة اللياقة البدنية المنزلية بأكملها. هذا السياق القائم على البيانات يؤكد أن تركيز المشروع على نهج رقمي أولاً هو قرار سليم استراتيجياً ومتوافق مع اتجاهات السوق السائدة^[8].

جغرافياً، تبرز أمريكا الشمالية باستمرار كسوق مهيمن، حيث تستحوذ على حصة تتراوح من 37.19% إلى 43.0%^[6]. تُعزى هذه الريادة إلى الوعي الصحي العالي لدى المستهلكين، والدخل المتاح الكبير، وثقافة قوية في تبني التكنولوجيا، مما يخلق سوقاً أولياً مثالياً لتطبيقات اللياقة البدنية المبتكرة. يوفر تجميع هذه الإحصائيات، كما هو مفصل في الجدول 2.1، أساساً كمياً قوياً لجدوى تطوير تطبيقات اللياقة البدنية وأهميتها في السوق^{[13][8]}.

معدل النمو السنوي المركب (%)	الحجم المتوقع (بالدولار الأمريكي)	حجم السوق لعام 2024 (بالدولار الأمريكي)	قطاع السوق
6.32	19.79 مليار بحلول 2032	12.26 مليار	معدات اللياقة المنزلية
8.2	18.03 مليار بحلول 2033	8.87 مليار	معدات اللياقة المنزلية
5.0	21.8 مليار بحلول 2033	14.07 مليار (تقدير 2024)	معدات اللياقة المنزلية
8.61	13.09 مليار بحلول 2034	5.73 مليار	اللياقة التفاعلية

الجدول (1): توقعات سوق اللياقة البدنية المنزلية والتفاعلية العالمي (2024-2033)

3.2 محركات السوق الرئيسية والعوامل التكنولوجية المساعدة

يُعزى النمو المتسارع لسوق اللياقة البدنية الرقمية في المنزل إلى تضافر التحولات الاجتماعية والثقافية والابتكار التكنولوجي. ويعد فهم هذه المحركات أمراً بالغ الأهمية لوضع ميزات التطبيق وقيمتها المقترحة في سياقها الصحيح.

أولاً وقبل كل شيء، هناك زيادة عالمية في الوعي الصحي. إن الوعي المتزايد لدى المستهلكين بالفوائد الصحية طويلة الأمد للنشاط البدني المنتظم والمخاطر المرتبطة بأنماط الحياة الكسولة، مثل السمنة وأمراض القلب والسكري هو حافز أساسي لنمو السوق^[12]. هذا الوعي المتزايد يخلق طلباً أساسياً على حلول لياقة بدنية فعالة وسهلة الوصول.

ثانياً، تعطي أنماط حياة المستهلكين الحديثة أولوية متزايدة للراحة والمرونة والخصوصية. تشكل التحديات اللوجستية لعضويات الصالات الرياضية التقليدية، مثل وقت التنقل وساعات العمل الثابتة والأماكن المزدحمة، حواجز كبيرة للكثيرين.^{[11][9]} تعالج اللياقة البدنية في المنزل هذه المشاكل مباشرة من خلال توفير القدرة على ممارسة الرياضة في أي وقت وفي أي مكان.

ويتضخم هذا الطلب أيضًا بسبب الرغبة المتزايدة في الحصول على تجارب تمرين مخصصة. يتجه المستهلكون بعيدًا عن البرامج العامة ذات المقاس الواحد للجميع ونحو الحلول التي تدعمها التكنولوجيا والتي يمكن تخصيصها لتناسب أهدافهم ومستويات لياقتهم وتقدمهم المحدد^{[8][12]}.

ثالثًا، يعمل التقدم التكنولوجي كحسم لهذا السوق. أدى الانتشار الواسع للهواتف الذكية، إلى جانب الاتصال بالإنترنت، إلى إنشاء البنية التحتية اللازمة لتقديم محتوى لياقة بدنية غني وتفاعلي مباشرة للمستخدمين. كما أدى ظهور التكنولوجيا القابلة للارتداء، مثل الساعات الذكية وأجهزة تتبع اللياقة البدنية، إلى زيادة دمج الأدوات الرقمية في حياة المستخدمين اليومية، مما أدى إلى تطبيع ممارسة تتبع النشاط البدني والبيانات الصحية^{[3][4]}. وقد أدى ذلك إلى نمو "منتجات اللياقة البدنية المتصلة" التي تسمح للمستخدمين بمراقبة أدائهم وجمع البيانات والتفاعل مع تطبيقات الهاتف المحمول في حلقة تغذية راجعة سلسلة. من الواضح أن السوق ينضج ليتجاوز مجرد تقديم المحتوى؛ فالنجاح الآن يتوقف على فهم متطور لتكامل البيانات، ومشاركة المستخدم، والقدرة على توفير تجربة تفاعلية وتكيفية حقيقية^{[7][10]}.

4.2 مكانة VexaFit الاستراتيجية

ضمن هذا السوق الواسع والمتطور، يتم وضع هذا تطبيق بشكل مدروس لخدمة شريحة محددة وغالبًا ما تكون غير مخدومة: الأفراد الذين يرغبون في الحفاظ على صحتهم ولياقتهم البدنية في المنزل دون الحاجة إلى مدربين محترفين أو معدات باهظة الثمن أو تمارين معقدة ومخيفة. يضع هذا التركيز تطبيقنا المراد تطويره مباشرة ضمن قطاعات التطبيقات "المنزلية" و"السكنية"، والتي تم تحديدها على أنها تمتلك أكبر حصة من الإيرادات في سوق اللياقة البدنية التفاعلية في عام 2024^{[2][6]}.

تتجذر القيمة الأساسية المقترحة للتطبيق في البساطة وإمكانية الوصول. من خلال استهداف المستخدمين الذين قد يكونون مبتدئين، أو لديهم ميزانية محدودة، أو مقيدون ببيئتهم المادية (مثل شقة سكن شبابي)، يعالج تطبيقنا العديد من الحواجز الرئيسية للدخول التي قد تتجاهلها المنصات الأكثر تعقيدًا أو التي تعتمد على المعدات. هذا النهج المرتكز على المستخدم، والذي يعطي الأولوية للتوجيه المباشر والروتينات التي تركز على الاعتماد على وزن الجسم فقط دون الحاجة لمعدات.

5.2 تحديات في اللياقة البدنية المنزلية غير الخاضعة للرقابة

بينما يتميز سوق اللياقة المنزلية بالقوة، فإن الانتقال من بيئة الصالة الرياضية المنظمة إلى بيئة منزلية غير خاضعة للرقابة يمثل مجموعة فريدة من التحديات للمستخدم النهائي. غالبًا ما تكون هذه العقبات أقل ارتباطًا بالقدرة البدنية وأكثر ارتباطًا بالحواجز النفسية والبيئية والمعلوماتية^[8]. من خلال تحديد هذه المشاكل بدقة، سيؤسس هذا القسم للضرورة الوظيفية للتطبيق كحل للصعوبات الواقعية التي تواجهها جمهوره المستهدف.

6.2 الحواجز النفسية والتحفيزية

العقبة الأهم في اللياقة المنزلية هي الحفاظ على الدافع والالتزام على المدى الطويل. المنزل هو مساحة متعددة الأغراض، غالبًا ما ترتبط نفسيًا بالعمل والاسترخاء والحياة الأسرية، مما يجعل من الصعب تنمية الطاقة المركزة المطلوبة للتمرين. بدون المحفزات الخارجية الكامنة في بيئة الصالة الرياضية مثل الفصول المجدولة، والاستثمار المالي، ووجود أشخاص آخرين يمكن أن يتضاءل الدافع الداخلي لممارسة الرياضة. كثيرًا ما يبلغ المستخدمون عن "رغبة متناقصة" وصعوبة في التحول من عقلية العمل أو الراحة إلى عقلية التمرين داخل نفس المساحة المادية^{[6][2]}. يمكن أن يؤدي هذا النقص في بيئة اللياقة البدنية المخصصة إلى ارتفاع معدلات التخلي عن تطبيقات اللياقة البدنية. تشير الأبحاث إلى أن الملل والرتابة من المساهمين الرئيسيين في هذا الانقطاع؛ يحتاج المستخدمون إلى التنوع والتجديد في روتيناتهم ليظلوا مهتمين وملتزمين.^{[12][7]}

المشكلة الأساسية، إذن، ليست نقصًا جوهريًا في الرغبة في التمتع بالصحة، بل غياب هيكل دعم للتغلب على الرهاب من مغادرة البيئة المنزلية. توفر الصالة الرياضية التقليدية هذا الهيكل من خلال تصميمها المادي، والمساءلة الاجتماعية، والبرمجة المجدولة. لكي ينجح تطبيقنا، يجب عليه تكرار هيكل الدعم هذا رقميًا من خلال توفير محتوى جذاب، ومسارات تقدم واضحة، وردود فعل تحفيزية تقلل بشكل فعال من الاحتكاك النفسي لبدء وإكمال التمرين في المنزل.^{[13][4]}

7.2 القيود البيئية واللوجستية

بالإضافة إلى التحديات النفسية، تفرض البيئة المادية للمنزل قيودًا عملية يمكن أن تعيق قدرة المستخدم على ممارسة الرياضة بفعالية. يعيش العديد من المستخدمين المحتملين، لا سيما في المناطق الحضرية، في شقق صغيرة ذات مساحة مفتوحة محدودة. وهذا يجعل التمارين التي تتطلب مساحة كبيرة للحركة أو معدات متخصصة غير عملية أو مستحيلة. لذلك يركز تطبيقنا على "بدون... تمارين معقدة"، ويتضمن مكتبة من التمارين التي يمكن أداؤها بأقل قدر من المعدات أو بدونها، يعالج هذا القيد الحاسم مباشرة^{[6][2]}.

علاوة على ذلك، يعد المنزل مساحة مشتركة، غالبًا ما تكون مليئة بالمشتتات من أفراد الأسرة أو الحيوانات الأليفة أو نداء الأعمال المنزلية المستمر. يمكن أن يؤدي نقص الخصوصية إلى شعور المستخدمين بالخجل أو المقاطعة بشكل متكرر، مما يكسر تركيزهم ويقلل من جودة التمرين. تتطلب هذه العوامل البيئية تمارين فعالة وجذابة ويمكن إكمالها في فترات قصيرة ومركزة. المستخدم المستهدف لتطبيقنا ليس مجرد مبتدئ، بل هو بشكل أدق فرد "محدود الموارد" مقيد بالوقت والمساحة والمعدات والمعرفة المتخصصة. وبالتالي، فإن القيمة المقترحة للتطبيق متجذرة بعمق في قدرته على تقديم حل لياقة فعال يحترم هذه القيود الواقعية ويتكيف معها^{[12][10]}.

الفصل الثالث

الدراسة المرجعية

يُعرض هذا الفصل الأبحاث المرتبطة بالعمل المقدم.

1.3 مقدمة

يهدف هذا الفصل إلى وضع هذا المشروع في سياقه الصحيح ضمن المشهد الحالي لتطبيقات اللياقة البدنية والتقنيات المستخدمة في تطويرها. من خلال إجراء دراسة مرجعية شاملة، نسعى إلى تحقيق فهم للبيئة التنافسية، واستخلاص الدروس من نجاحات وتحديات التطبيقات الأخرى في السوق.

2.3 بعض التطبيقات الشائعة

1.2.3 رائد المحتوى المجاني (Nike Training Club):

- **نظرة عامة على المنصة:** يعد Nike Training Club قوة مهيمنة في السوق، ويرجع ذلك أساسًا إلى مكتبته الواسعة من محتوى التمارين عالي الجودة، والذي يقدمه مجانًا تمامًا. يوفر التطبيق مجموعة واسعة من تمارين الفيديو عند الطلب عبر فئات عديدة، بما في ذلك تدريب القوة واليوغا والتمارين المنقطعة عالية الكثافة (HIIT) والحركة، بقيادة مدربين معتمدين وغالبًا ما يظهر فيه رياضيون مشهورون. إحدى الميزات الرئيسية هي برامجه المنظمة متعددة الأسابيع التي تزداد شدتها تدريجيًا، مما يجعله نقطة انطلاق ممتازة للمبتدئين. يتبنى NTC أيضًا نهجًا شاملاً من خلال تقديم محتوى صحي حول التغذية والعقلية والتعافي.^{[6][8]}
- **الوضع التنافسي:** تكمن القوة الأساسية لـ NTC في القيمة التي لا تضاهي لعرضه المجاني، مدعومًا بمصادقية علامة Nike التجارية وشهرتها. إنه مناسب بشكل خاص للمستخدمين الذين لا يحتاجون إلى معدات ويبحثون عن محتوى فيديو موجه. ومع ذلك، تكمن نقطة ضعفه الرئيسية في افتقاره إلى التخصيص العميق؛ فبرامج التمرين محددة مسبقًا ولا يمكن للمستخدم تخصيصها. علاوة على ذلك، انتقد بعض المستخدمين الإصدارات الحديثة من التطبيق لأنها أصبحت "متضخمة" بتعليقات المدربين المفرطة، مما قد يتعارض مع تجربة التمرين.^{[2][8]}
- **الآثار الاستراتيجية لـ VexaFit:** يمثل NTC منافسًا مباشرًا هائلًا لأي مستخدم يبحث عن تمارين منزلية مجانية. لا يمكن لتطبيقنا التنافس على حجم المحتوى أو قوة العلامة التجارية. بدلاً من ذلك، تكمن فرصته للتمايز في توفير تجربة مستخدم متفوقة تركز على البساطة والتخصيص. بينما يقدم NTC برامج جاهزة، فإن الميزة الأساسية لتطبيقنا

هي قدرة المستخدمين على بناء وتعديل تمارينهم الخاصة من الألف إلى الياء. هذا يلبي مباشرة حاجة المستخدم إلى الاستقلالية، ويقدم تجربة أكثر انسيابية وأقل توجيهًا من NTC.

2.2.3 تطبيق Peloton تجربة مجتمعية بقيادة المدربين:

- **نظرة عامة على المنصة:** اكتسب تطبيق Peloton حصة سوقية كبيرة من خلال نجاحه في نقل طاقة ومجتمع فصول اللياقة البدنية الراقية إلى شكل رقمي. يتمثل عرضه الأساسي في فصول حية ومسجلة عالية الطاقة بقيادة مدربين يتمتعون بشخصية جذابة ومحفزة. تشتهر المنصة بتعزيز شعور قوي بالمجتمع من خلال ميزات تفاعلية مثل لوحة المتصدرين المباشرة.
- **الوضع التنافسي ونقاط القوة والضعف:** الميزة التنافسية الرئيسية لـ Peloton هي تركيزه على الجوانب التحفيزية والاجتماعية للياقة البدنية. أما نقطة ضعفه، من منظور المستخدم المستهدف لتطبيقنا، فهي نموذج القائم على الاشتراك المدفوع، وتركيزه على تجربة تنافسية قد لا تناسب المبتدئين أو من يفضلون ممارسة الرياضة بخصوصية.
- **الآثار الاستراتيجية على VexaFit:** يستهدف Peloton و تطبيقنا المقترح فئات مختلفة من المستخدمين. يبحث مستخدم Peloton عن دافع خارجي من المدربين ومجتمع تنافسي. في المقابل، المستخدم الذي نستهدفه يكون فردًا ذا توجه ذاتي يفضل رحلة لياقة بدنية أكثر هدوءًا وخصوصية. لا يحاول تكرار بيئة الفصول عالية الطاقة، بل يقدم أداة للتخطيط والتنفيذ الشخصي.

3.2.3 متتبع التغذية المعتمد على البيانات MyFitnessPal:

- **نظرة عامة على المنصة:** يُعد MyFitnessPal رائدًا في السوق، لكن كفاءته الأساسية تكمن في تتبع التغذية بدلاً من توجيه التمارين. ميزته الرئيسية هي قاعدة بيانات الأطعمة الكبيرة بشكل استثنائي ومجموعة من الأدوات القوية لتسجيل السرعات الحرارية.^{[6][11]}
- **الوضع التنافسي ونقاط القوة والضعف:** قاعدة بيانات MyFitnessPal التي لا مثيل لها تجعله التطبيق المفضل لأي شخص يركز على إدارة النظام الغذائي. ومع ذلك، فإن توجيهاته للتمارين ومكتبة التمارين ليست قوية أو جذابة مثل تلك الموجودة في تطبيقات اللياقة البدنية المخصصة.^{[11][12]}
- **الآثار الاستراتيجية على VexaFit:** لا يُعد MyFitnessPal منافسًا مباشرًا من حيث تقديم التمارين. ومع ذلك، تُظهر شعبيته الهائلة طلبًا قويًا في السوق على تتبع البيانات ومراقبة التقدم. هذا يشير إلى أن ميزة تطبيقنا لتتبع تاريخ التمارين، والأرقام القياسية الشخصية، ومقاييس الأداء الأخرى ستكون مكونًا ذا قيمة عالية. يثبت MyFitnessPal أن المستخدمين يتم تحفيزهم من خلال رؤية بياناتهم وفهم تقدمهم، وهو مبدأ يمكن لتطبيقنا المقترح تطبيقه خصيصًا على التمارين.

3.3 خلاصة الدراسة المرجعية

يكشف هذا التحليل عن تجزئة واضحة للسوق بناءً على دافع المستخدم ونموذج العمل. يلي NTC "المستخدم الباحث عن محتوى مجاني موجه"، بينما يلي Peloton "المستخدم الاجتماعي التنافسي"، ويخدم MyFitnessPal "المستخدم المهتم بمتابعة البيانات الغذائية".

تتمثل الفرصة هنا في استهداف شريحة محددة: "المبتدئ الموجه ذاتيًا"، وهو مستخدم يقدر البساطة، والاستقلالية، والوضوح. يوضح المشهد التنافسي أنه في حين أن المحتوى المجاني له قوة تنافسية هائلة، إلا أنه لا يضمن تجربة مستخدم مثالية للجميع. شكاوى المستخدمين حول افتقار NTC للتخصيص تفتح الباب أمام تطبيقات مثل تطبيقنا المقترح الذي لا ينافس على كمية الميزات، بل على جودة وتركيز تجربة المستخدم.

الميزة	VexaFit (المقترح)	Nike Training Club	تطبيق Peloton	MyFitnessPal
التركيز الأساسي	تمارين منزلية بسيطة وقابلة للتخصيص	تمارين فيديو موجهة وبرامج تدريبية	فصول حية ومسجلة بقيادة مدربين	تتبع التغذية والسعرات الحرارية
نموذج التسعير	مجاني	مجاني	اشتراك مدفوع	مجاني مع ميزات مدفوعة
إنشاء تمرين مخصص	نعم (ميزة أساسية)	لا	لا	محدود
برامج موجهة	نعم (محددة مسبقًا)	نعم (متعددة الأسابيع)	نعم (برامج وتحديات)	محدود

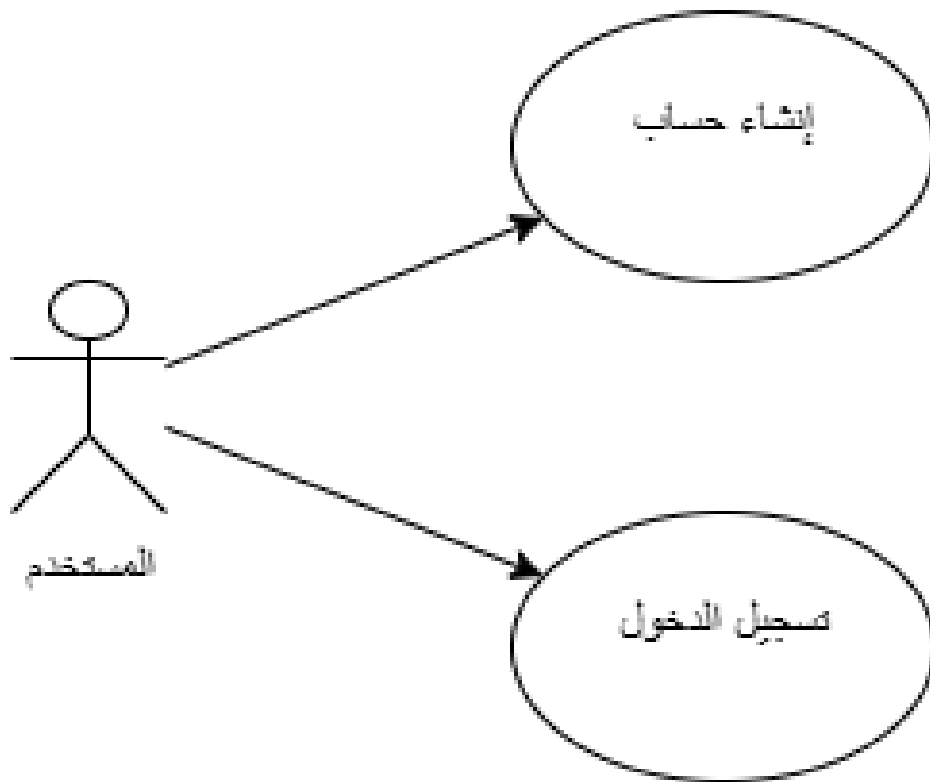
الجدول (2): مقارنة بين بعض التطبيقات المشابهة للتطبيق المقترح

الفصل الرابع

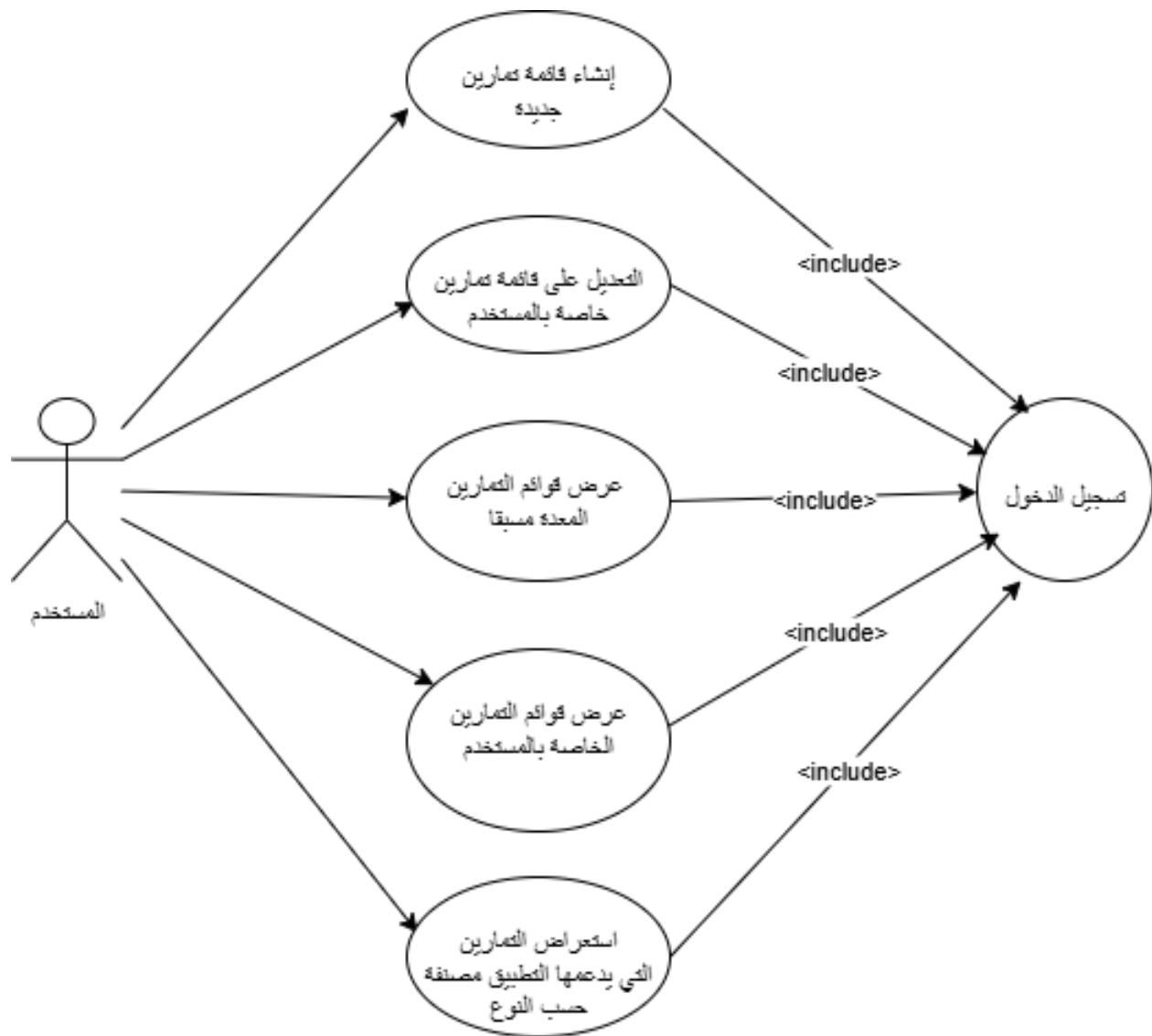
الدراسة التحليلية

يوضح هذا الفصل عملية تحليل النظام ودراسة متطلباته.

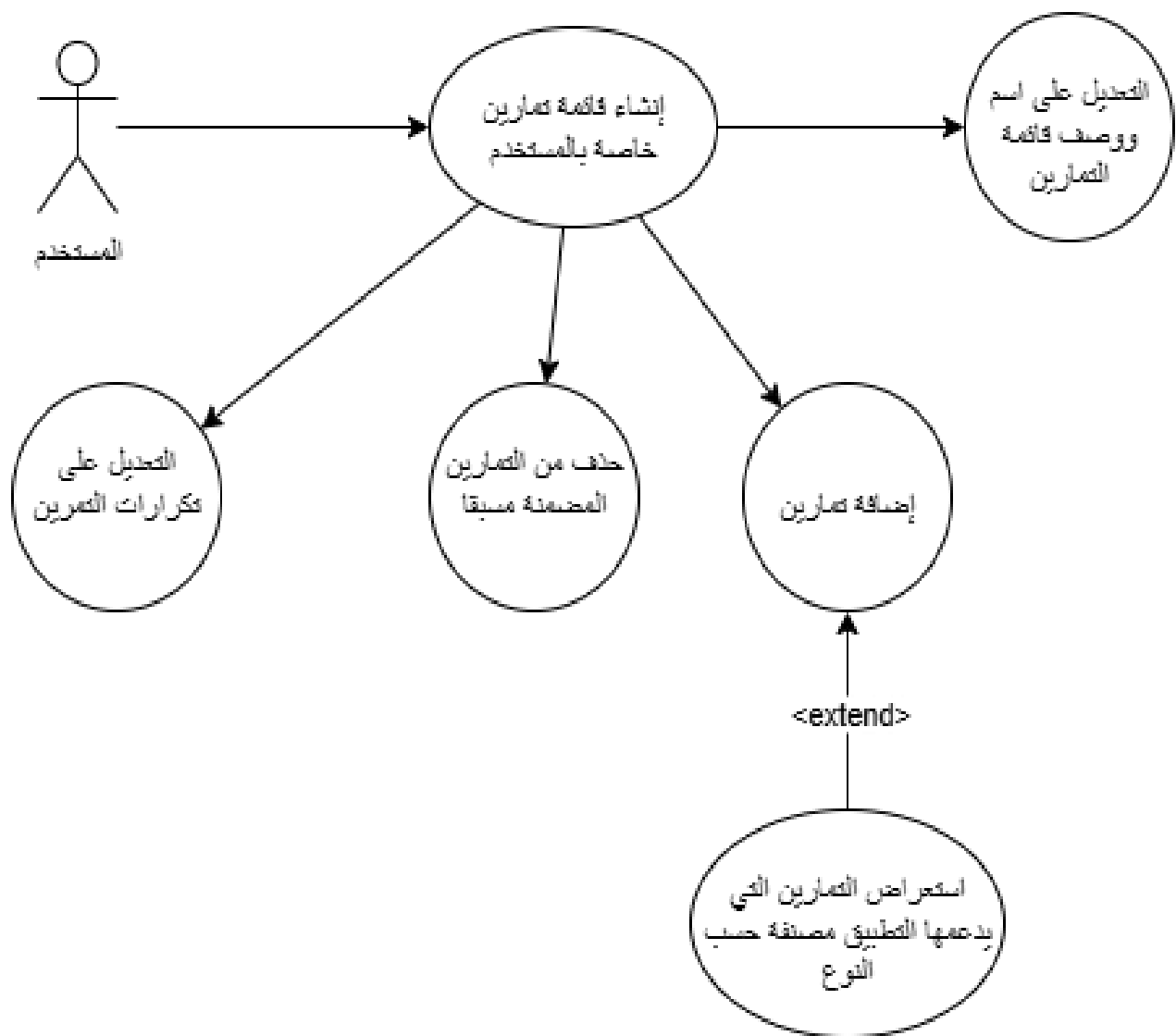
1.4 مخططات حالات الاستخدام للمستخدم



الصورة (1): حالات استخدام إدارة حسابات المستخدمين



الصورة (2): حالات استخدام إدارة عمليات النظام للمستخدم



الصورة (3): حالات استخدام إدارة عمليات النظام للمستخدم

2.4 السرد النصي لحالات الاستخدام للمستخدم

1.2.4 إنشاء حساب جديد

اسم الحالة: إنشاء حساب جديد	
الوصف Description	يقوم المستخدم بإنشاء حساب جديد خاص به.
الفاعلين Actors	المستخدم
الشروط السابقة Precondition	لا يوجد.
الشروط اللاحقة Postcondition	تم إنشاء حساب جديد خاص بالمستخدم ضمن النظام.

الجدول (3): السرد النصي لحالة استخدام إنشاء حساب جديد للمستخدم

سير الأحداث

السيناريو الأساسي الناجح

النظام	المستخدم
	1. يطلب إنشاء حساب جديد.
2. يطلب النظام المعلومات التالية: - البريد الإلكتروني الخاص بالمستخدم. - كلمة المرور. - تأكيد كلمة المرور. - اسم المستخدم.	
	3. يحدد المستخدم المعلومات المطلوبة ويطلب تأكيد العملية.
4. يتحقق النظام من صحة المعلومات ويعيد للمستخدم رسالة توضح إنهاء العملية.	

الجدول (4): السيناريو الناجح لحالة استخدام إنشاء حساب جديد للمستخدم

المسارات البديلة

لا يوجد.

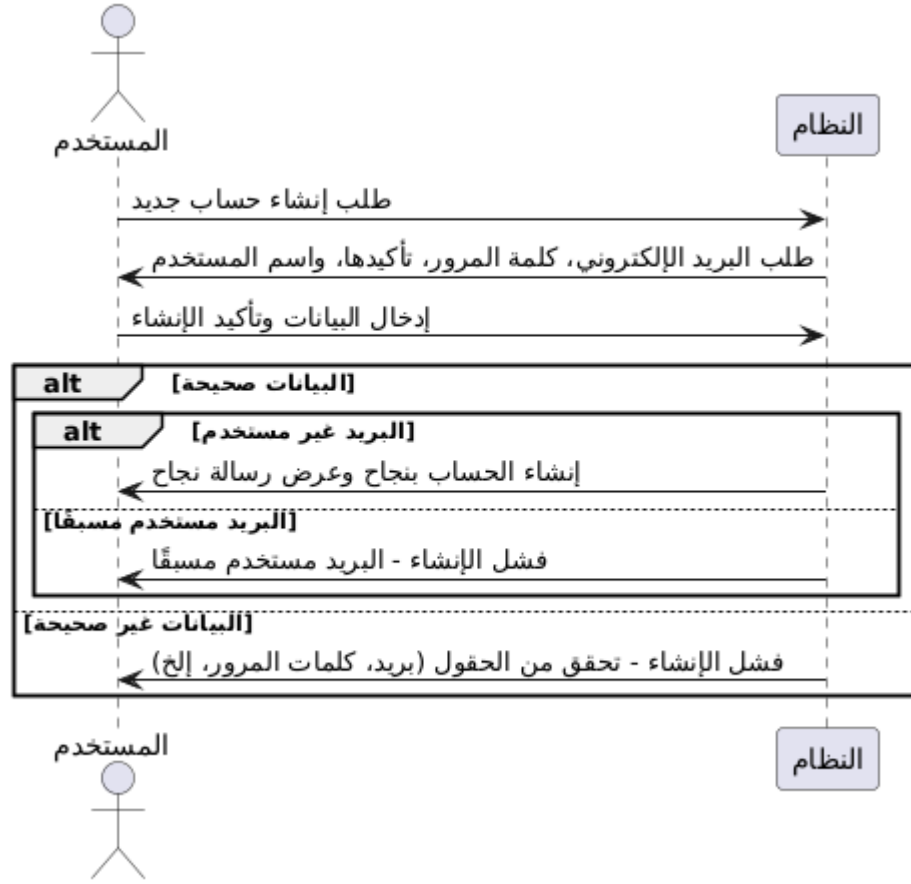
مسارات الأخطاء

- **E1:** في المرحلة رقم 4 في حال تحققت إحدى الحالات التالية:
 - البريد الإلكتروني غير صالح.
 - كلمة المرور ضعيفة (لا تحقق الشروط المطلوبة).
 - كلمة المرور لا تتطابق مع حقل تأكيد كلمة المرور.
 - في حال تحققت إحدى الحالات السابقة، يتم استبدال الخطوة الرابعة في السيناريو الأساسي بالخطوة التالية:

4. يعيد النظام رسالة توضح سبب الخطأ ويطلب تحديد المعلومات من جديد.

- **E2:** في المرحلة رقم 4 في حال تحققت الحالة التالية:
 - البريد الإلكتروني مسجل مسبقاً ضمن النظام.
 - في حال تحققت الحالة السابقة، يتم استبدال الخطوة الرابعة في السيناريو الأساسي بالخطوة التالية:

4. يعيد النظام رسالة توضح سبب الخطأ ويطلب تحديد المعلومات من جديد.



الصورة (4): مخطط SSD لحالة الاستخدام إنشاء حساب جديد

2.2.4 تسجيل الدخول

اسم الحالة: تسجيل الدخول	
الوصف Description	يقوم المستخدم بتسجيل الدخول إلى حسابه.
الفاعلين Actors	المستخدم.
الشروط السابقة Precondition	يجب أن يمتلك المستخدم حسابا فعالا ومسجلا في النظام.
الشروط اللاحقة Postcondition	تم تسجيل دخول المستخدم بنجاح إلى النظام.

الجدول (5): السرد النصي لحالة استخدام تسجيل الدخول للمستخدم

النظام	المستخدم
	1. يطلب تسجيل الدخول.
2. يطلب النظام المعلومات التالية: - البريد الإلكتروني الخاص بالمستخدم. - كلمة المرور.	
	3. يحدد المستخدم المعلومات المطلوبة ويطلب تأكيد العملية.
4. يتحقق النظام من صحة المعلومات ويصادق على المستخدم ثم يعرض الواجهة الرئيسية للتطبيق.	

الجدول (6): السيناريو الناجح لحالة استخدام تسجيل الدخول للمستخدم

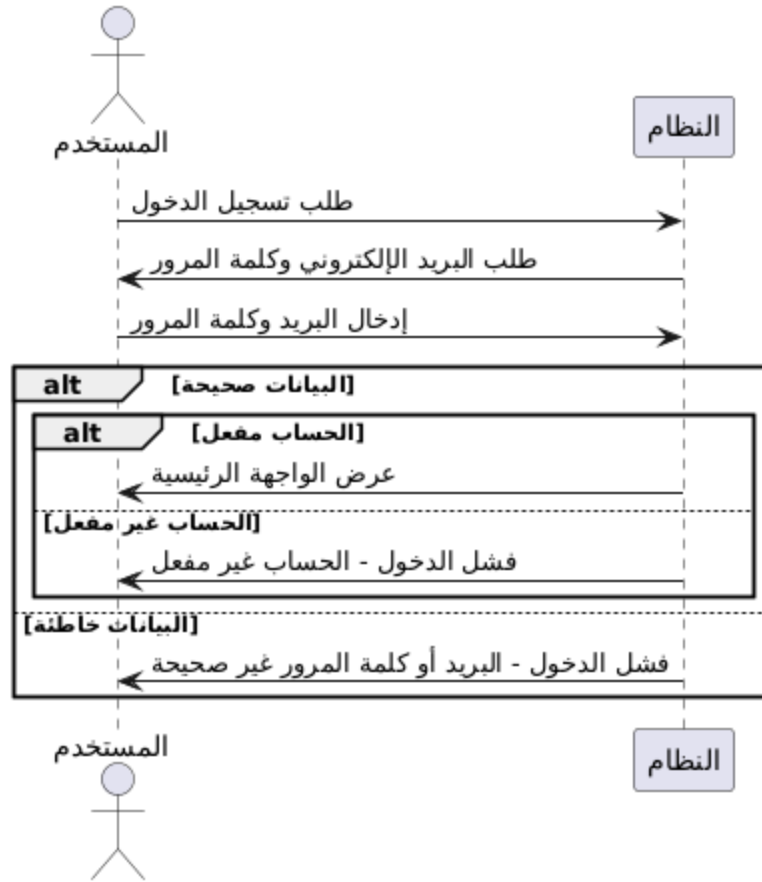
المسارات البديلة

لا يوجد.

مسارات الأخطاء

- E1: في المرحلة رقم 4 في حال تحققت إحدى الحالات التالية:
 - البريد الإلكتروني أو كلمة المرور غير صحيحة.
 - حساب المستخدم غير نشط.
 - في حال تحققت إحدى الحالات السابقة، يتم استبدال الخطوة الرابعة في السيناريو الأساسي بالخطوة التالية:

1. يعيد النظام رسالة توضح سبب الخطأ ويطلب إدخال المعلومات من جديد.



الصورة (5): مخطط SSD لحالة الاستخدام تسجيل دخول المستخدم

3.2.4 إنشاء قائمة تمارين خاصة بالمستخدم

اسم الحالة: إنشاء قائمة تمارين خاصة بالمستخدم	
الوصف Description	يقوم المستخدم بإنشاء قائمة تمارين جديدة ومخصصة له.
الفاعلين Actors	المستخدم
الشروط السابقة Precondition	يجب أن يكون المستخدم قد قام بتسجيل الدخول إلى النظام.
الشروط اللاحقة Postcondition	تم إنشاء قائمة التمارين الجديدة وحفظها في حساب المستخدم.

الجدول (7): السرد النصي لحالة استخدام إنشاء قائمة تمارين خاصة بالمستخدم

النظام	المستخدم
	1. يطلب إنشاء قائمة تمارين جديدة.
2. يطلب النظام المعلومات التالية: - اسم لقائمة التمارين. - وصف لقائمة التمارين.	
	3. يحدد المستخدم المعلومات المطلوبة ويطلب تأكيد العملية.
4. يقوم النظام بحفظ قائمة التمارين الفارغة ويعرض للمستخدم شاشة تفاصيل القائمة لإضافة التمارين.	

الجدول (8): السيناريو الناجح لحالة استخدام إنشاء قائمة تمارين خاصة بالمستخدم

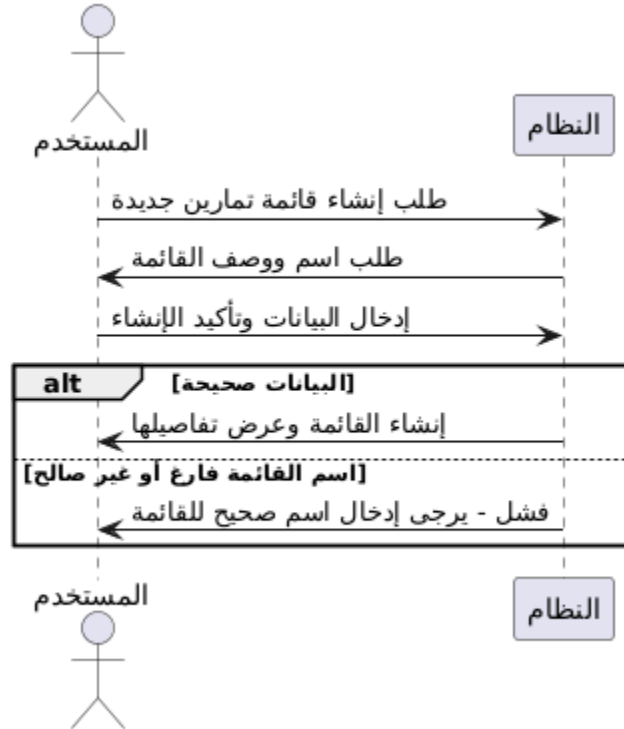
المسارات البديلة

لا يوجد.

مسارات الأخطاء

- E1: في المرحلة رقم 4 في حال كانت البيانات المدخلة غير صالحة (مثل اسم فارغ):

1. يعرض النظام رسالة خطأ ويطلب من المستخدم إدخال معلومات صحيحة.



الصورة (6): مخطط SSD لحالة الاستخدام إنشاء قائمة تمارين خاصة بالمستخدم

4.2.4 إضافة تمرين إلى قائمة التمارين الخاصة بالمستخدم

اسم الحالة: إضافة تمرين إلى قائمة التمارين الخاصة بالمستخدم	
الوصف Description	يقوم المستخدم بإضافة تمرين من مكتبة التمارين إلى إحدى قوائمها الخاصة.
الفاعلين Actors	المستخدم.
الشروط السابقة Precondition	يجب أن يكون المستخدم قد قام بإنشاء قائمة تمارين خاصة به.
الشروط اللاحقة Postcondition	تم إضافة التمرين المحدد إلى قائمة التمارين الخاصة بالمستخدم.

الجدول (9): السرد النصي لحالة استخدام إضافة تمرين إلى قائمة التمارين الخاصة بالمستخدم

سير الأحداث

السيناريو الأساسي الناجح

النظام	المستخدم
	1. يختار المستخدم قائمة التمارين الخاصة به ويطلب إضافة تمرين جديد.
2. يستعرض النظام قائمة بالتمارين المتاحة مع إمكانية البحث.	
	3. يختار المستخدم التمرين المراد إضافته.
4. يطلب النظام من المستخدم تحديد تفاصيل التمرين (عدد الجولات، التكرارات، الوزن، المدة).	
	5. يدخل المستخدم التفاصيل المطلوبة ويؤكد الإضافة.
1. يقوم النظام بحفظ التمرين وتفاصيله ضمن قائمة التمارين ويعرض القائمة المحدثة.	

الجدول (10): السيناريو الناجح لحالة استخدام إضافة تمرين إلى قائمة التمارين الخاصة بالمستخدم

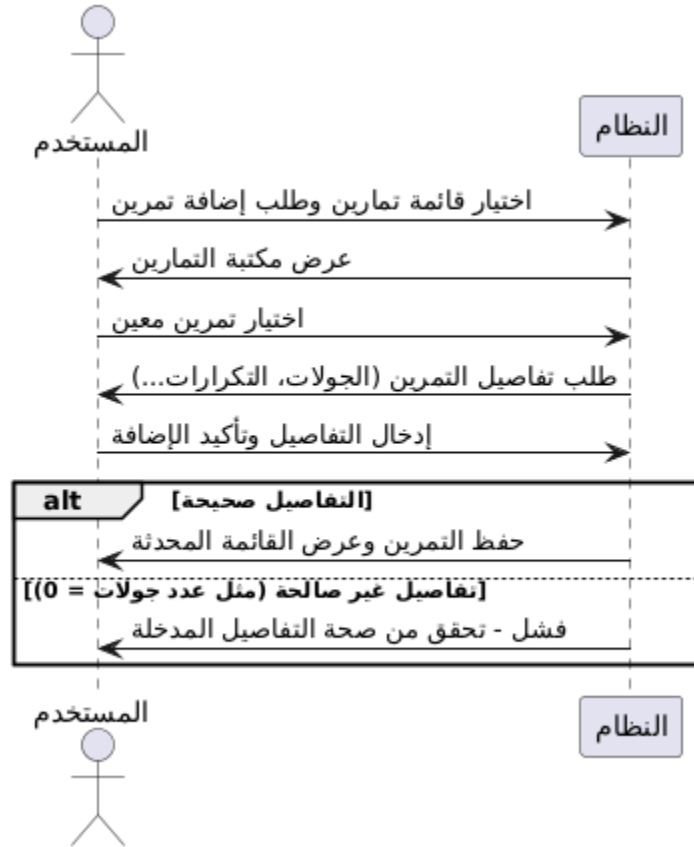
المسارات البديلة

لا يوجد.

مسارات الأخطاء

- E1: في المرحلة رقم 6 في حال كانت تفاصيل التمرين غير صالحة (مثل عدد جولات يساوي صفر):

1. يعرض النظام رسالة خطأ ويطلب من المستخدم إدخال معلومات صحيحة.



الصورة (7): مخطط SSD لحالة الاستخدام إضافة تمرين إلى قائمة التمارين الخاصة بالمستخدم

5.2.4 عرض قوائم التمارين (المعدة مسبقاً والخاصة)

اسم الحالة: عرض قوائم التمارين (المعدة مسبقاً والخاصة)	
الوصف Description	يقوم المستخدم باستعراض قوائم التمارين المعدة مسبقاً من قبل النظام أو القوائم الخاصة التي أنشأها.
الفاعلين Actors	المستخدم.
الشروط السابقة Precondition	يجب أن يكون المستخدم قد قام بتسجيل الدخول إلى النظام.
الشروط اللاحقة Postcondition	تم عرض قوائم التمارين للمستخدم.

الجدول (11): السرد النصي لحالة استخدام عرض قوائم التمارين (المعدة مسبقاً والخاصة)

النظام	المستخدم
	1. ينتقل المستخدم إلى شاشة التمارين.
2. يعرض النظام واجهة تحتوي على قسمين "قوائم التمارين المعدة مسبقاً" و "قوائم التمارين الخاصة بي."	
	3. يختار المستخدم إحدى القوائم لعرض تفاصيله..
4. يعرض النظام تفاصيل القائمة المختارة بما في ذلك التمارين الموجودة فيها.	

الجدول (12): السيناريو الناجح لحالة استخدام عرض قوائم التمارين (المعدة مسبقاً والخاصة)

المسارات البديلة

لا يوجد.

6.2.4 استعراض التمارين التي يدعمها التطبيق مصنفة حسب النوع

اسم الحالة: استعراض التمارين التي يدعمها التطبيق مصنفة حسب النوع	
الوصف Description	يقوم المستخدم بتصفح مكتبة التمارين الكاملة المتاحة في التطبيق.
الفاعلين Actors	المستخدم.
الشروط السابقة Precondition	يجب أن يكون المستخدم قد قام بتسجيل الدخول إلى النظام.
الشروط اللاحقة Postcondition	تم عرض مكتبة التمارين للمستخدم.

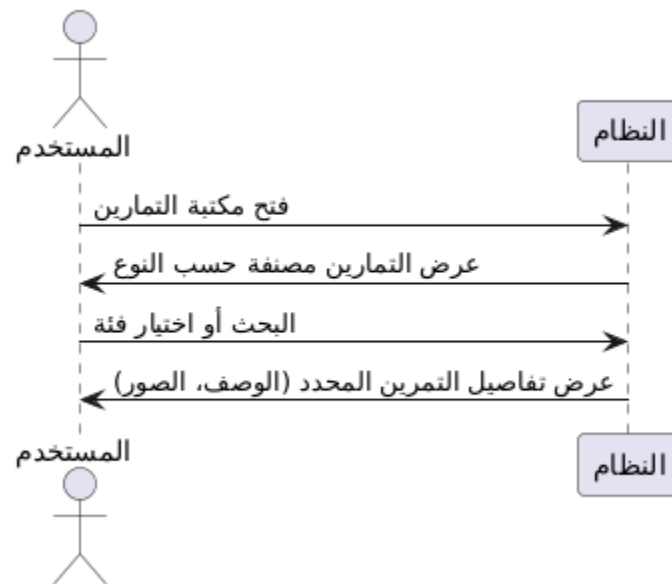
الجدول (13): السرد النصي لحالة استخدام استعراض التمارين

المستخدم	النظام
1. ينتقل المستخدم إلى شاشة مكتبة التمارين.	
	2. يعرض النظام جميع التمارين مصنفة ضمن فئات (مثل: عضلات الصدر، تمارين كارديو).
3. يقوم المستخدم بالبحث عن تمرين معين أو يختار فئة لتصفحها، ثم يختار تمريناً محدداً.	
	4. يعرض النظام تفاصيل التمرين المحدد (صور، وصف).

الجدول (14): السيناريو الناجح لحالة استخدام استعراض التمارين

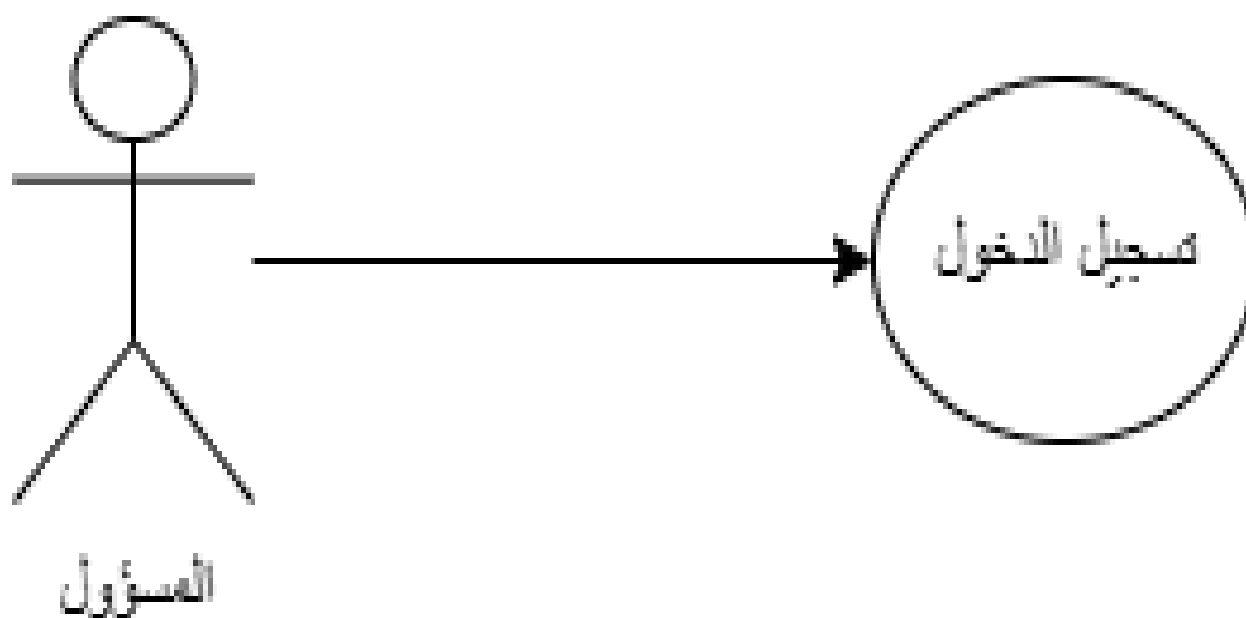
المسارات البديلة

لا يوجد.

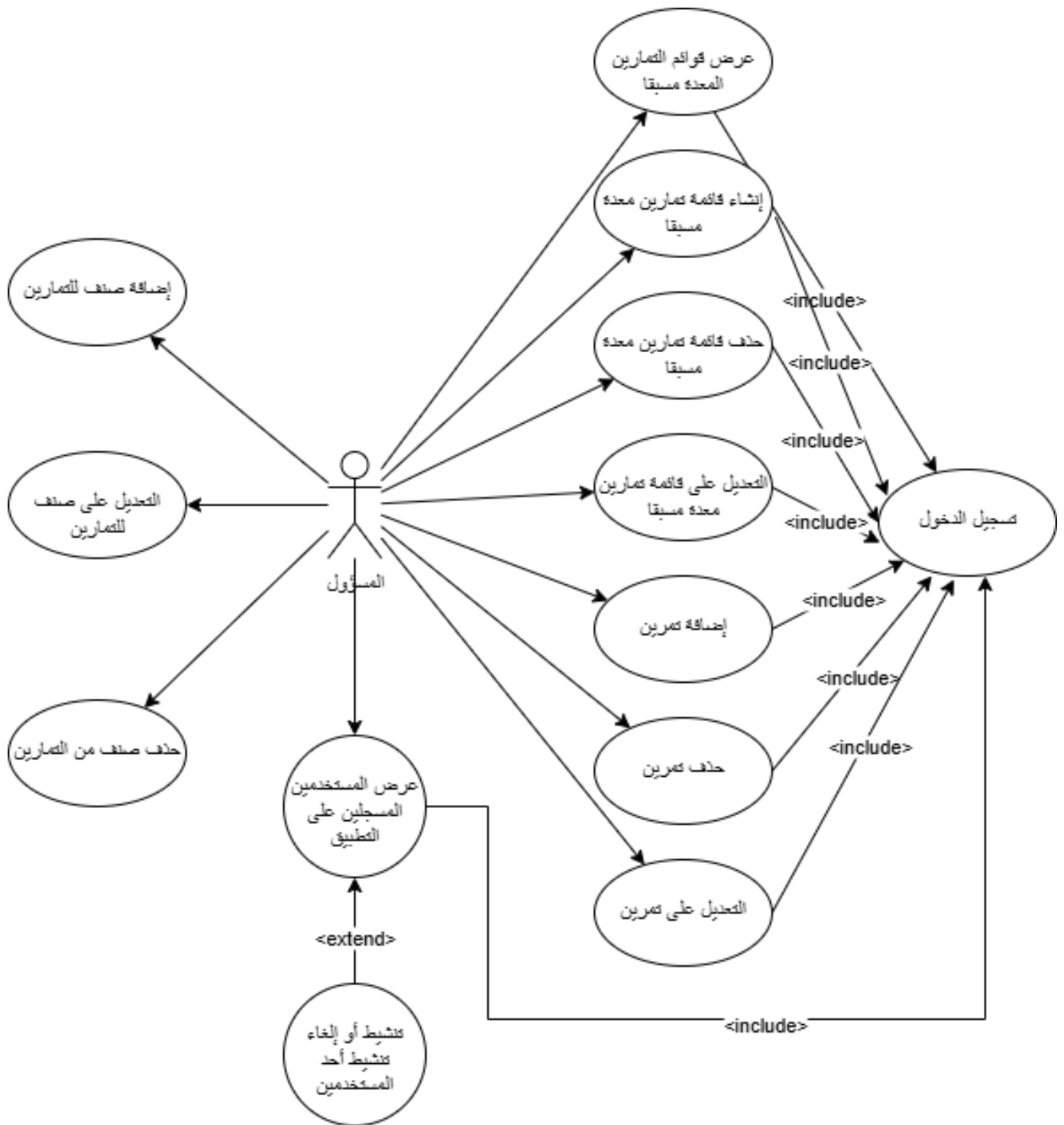


الصورة (8): مخطط SSD لحالة الاستخدام استعراض التمارين

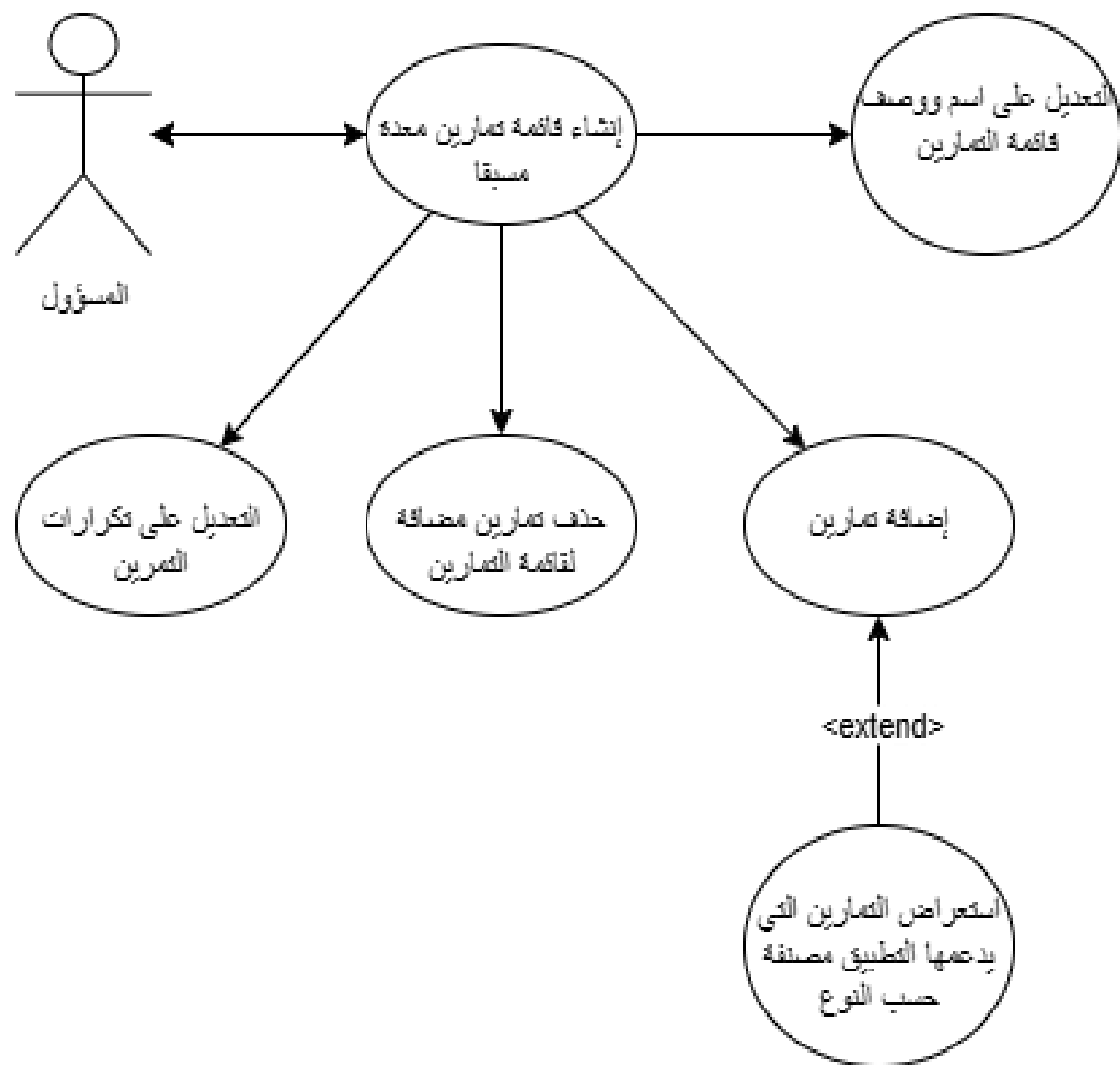
3.4 مخطط حالات الاستخدام للمسؤول



الصورة (9): حالات استخدام إدارة حسابات المسؤول



الصورة (10): حالات استخدام إدارة عمليات النظام للمسؤول



الصورة (11): حالات استخدام إدارة عمليات النظام للمسؤول

4.4 السرد النصي لحالات الاستخدام للمسؤول

1.4.4 عرض المستخدمين المسجلين وتغيير حالتهم

اسم الحالة: عرض المستخدمين المسجلين وتغيير حالتهم	
الوصف Description	يقوم المسؤول باستعراض قائمة بجميع المستخدمين المسجلين في التطبيق، مع إمكانية تفعيل أو إلغاء تفعيل حساباتهم.
الفاعلين Actors	المسؤول.
الشروط السابقة Precondition	يجب أن يكون المسؤول قد قام بتسجيل الدخول إلى لوحة التحكم.
الشروط اللاحقة Postcondition	يتم تحديث حالة المستخدم المحدد (نشط/غير نشط) في قاعدة البيانات.

الجدول (15): السرد النصي لحالة استخدام عرض المستخدمين المسجلين وتغيير حالتهم

سير الأحداث

السيناريو الأساسي الناجح

النظام	المستخدم
	1. يختار المسؤول قسم "إدارة المستخدمين" من لوحة التحكم.
2. يعرض النظام قائمة بجميع المستخدمين مع معلوماتهم الأساسية (الاسم، البريد الإلكتروني، الحالة الحالية).	
	3. يختار المسؤول مستخدماً محدداً ويضغط على زر "تغيير الحالة".
4. يقوم النظام بتحديث حالة المستخدم في قاعدة البيانات، ثم يعيد تحميل قائمة المستخدمين بالحالة المحدثة.	

الجدول (16): السيناريو الناجح لحالة استخدام عرض المستخدمين المسجلين وتغيير حالتهم

المسارات البديلة

لا يوجد.

2.4.4 إضافة صنف جديد للتمارين

اسم الحالة: إضافة صنف جديد للتمارين	
الوصف Description	يقوم المسؤول بإنشاء صنف (فئة) جديد للتمارين (مثل: تمارين كارديو، تمارين قوة).
الفاعلين Actors	المسؤول.
الشروط السابقة Precondition	يجب أن يكون المسؤول قد قام بتسجيل الدخول إلى لوحة التحكم.
الشروط اللاحقة Postcondition	يتم إنشاء الصنف الجديد وحفظه في قاعدة البيانات.

الجدول (17): السرد النصي لحالة استخدام إضافة صنف جديد للتمارين

سير الأحداث

السيناريو الأساسي الناجح

النظام	المستخدم
	1. يختار المسؤول قسم "إدارة الأصناف" ويضغط على زر "إضافة صنف جديد."
2. يعرض النظام نموذجاً لإدخال معلومات الصنف الجديد (اسم الصنف).	
	3. يدخل المسؤول اسم الصنف الجديد ويضغط على زر "حفظ."

4. يتحقق النظام من أن اسم الصنف غير مكرر، ثم يقوم بحفظ الصنف الجديد في قاعدة البيانات ويعرض رسالة تأكيد.	
--	--

الجدول (18): السيناريو الناجح لحالة استخدام إضافة صنف جديد للتمارين

المسارات البديلة

لا يوجد.

مسارات الأخطاء

• E1: في المرحلة رقم 4 في حال كان اسم الصنف فارغاً:

4. يعرض النظام رسالة خطأ تطلب من المسؤول إدخال اسم للصنف.

• E2: في المرحلة رقم 4 في حال كان اسم الصنف موجوداً مسبقاً:

4. يعرض النظام رسالة خطأ تفيد بأن اسم الصنف مكرر ويطلب إدخال اسم مختلف.

3.4.4 إضافة تمرين جديد

اسم الحالة: إضافة تمرين جديد	
الوصف Description	يقوم المسؤول بإضافة تمرين جديد إلى مكتبة التمارين في النظام.
الفاعلين Actors	المسؤول
الشروط السابقة Precondition	يجب أن يكون المسؤول قد قام بتسجيل الدخول إلى لوحة التحكم.
الشروط اللاحقة Postcondition	يتم حفظ التمرين الجديد مع كافة تفاصيله في قاعدة البيانات.

الجدول (19): السرد النصي لحالة استخدام إضافة تمرين جديد

النظام	المستخدم
	1. يختار المسؤول قسم "إدارة التمارين" ويضغط على زر "إضافة تمرين جديد."
2. يعرض النظام نموذجاً لإدخال تفاصيل التمرين (الاسم، الوصف، الصنف التابع له).	
	3. يقوم المسؤول بملء جميع الحقول المطلوبة ثم يضغط على زر "حفظ."
4. يقوم النظام بحفظ التمرين الجديد وربطه بالصنف المحدد ويعرض رسالة تأكيد بنجاح العملية.	

الجدول (20): السيناريو الناجح لحالة استخدام إضافة تمرين جديد

المسارات البديلة

لا يوجد.

مسارات الأخطاء

- E1: في المرحلة رقم 4 في حال كانت بعض الحقول الإلزامية فارغة (مثل اسم التمرين):

4. يعرض النظام رسالة خطأ توضح الحقول المطلوبة ويطلب من المسؤول إكمالها.

4.4.4 إنشاء قائمة تمارين معدة مسبقاً

اسم الحالة: إنشاء قائمة تمارين معدة مسبقاً	
الوصف Description	يقوم المسؤول بإنشاء قائمة تمارين جديدة ستكون متاحة لجميع مستخدمي التطبيق.
الفاعلين Actors	المسؤول.
الشروط السابقة Precondition	يجب أن يكون المسؤول قد قام بتسجيل الدخول إلى لوحة التحكم.
الشروط اللاحقة Postcondition	يتم حفظ قائمة التمارين الجديدة في قاعدة البيانات كقائمة معدة مسبقاً.

الجدول (21): السرد النصي لحالة استخدام إنشاء قائمة تمارين معدة مسبقاً

سير الأحداث

السيناريو الأساسي الناجح

النظام	المستخدم
1. يختار المسؤول قسم "إدارة قوائم التمارين" ويضغط على زر "إنشاء قائمة جديدة".	
2. يطلب النظام إدخال اسم ووصف لقائمة التمارين.	
3. يدخل المسؤول الاسم والوصف ويؤكد العملية.	
4. يقوم النظام بحفظ القائمة الفارغة ويحدث صفحة التمارين المعدة مسبقاً.	

الجدول (22): السيناريو الناجح لحالة استخدام إنشاء قائمة تمارين معدة مسبقاً

المسارات البديلة

لا يوجد.

الفصل الخامس

أسلوب العمل

يُعرض هذا الفصل المنهجية المقترحة لتصميم التطبيق.

1.5 مقدمة

يهدف هذا الفصل إلى توضيح المنهجية الهندسية التي تم اتباعها في تطوير مشروع VexaFit، والتي شكلت الجسر بين الأبحاث النظرية والتطبيق العملي. بعد إرساء الأساس المعرفي من خلال الدراسة النظرية والدراسة المرجعية، كان من الضروري اعتماد منهجية تطوير تضمن ترجمة هذه الأفكار والمتطلبات إلى منتج برمجي ناجح وعالي الجودة.

نظراً لطبيعة المشروع التي تتطلب المرونة والقدرة على التكيف مع المتطلبات المتغيرة واكتشاف أفضل الحلول بشكل تدريجي، تم اعتماد **المنهجية الرشيقية (Agile Methodology)** كإطار عمل أساسي. تسمح هذه المنهجية بتقسيم المشروع إلى دورات تطوير قصيرة ومتكررة (Sprints)، مما يتيح تسليم أجزاء وظيفية من النظام بشكل دوري، والحصول على تغذية راجعة مستمرة، وبالتالي تحسين المنتج بشكل تدريجي ومستمر بناءً على رؤى واضحة وملموسة.

2.5 نموذج التطوير المتبع

تم اختيار نموذج التطوير التزايد والتكراري (Iterative and Incremental Model) كآلية لتطبيق المنهجية الرشيقية. يركز هذا النموذج على بناء النظام قطعة قطعة، حيث يتم في كل دورة تطوير (Iteration)، والتي تُعرف باسم "Sprint"، تحليل وتصميم وتنفيذ واختبار مجموعة صغيرة ومحددة من الوظائف. تم تحديد مدة كل دورة تطوير بأسبوعين، مما يوفر إيقاعاً منتظماً للعمل ويتيح وقتاً كافياً لإنجاز مجموعة من الميزات القابلة للعرض.

مبررات اختيار هذا النموذج:

1. **المرونة والتكيف:** أظهرت الدراسة المرجعية أن سوق تطبيقات اللياقة البدنية ديناميكي ومتغير. يسمح النموذج التكراري بإدخال تعديلات وتحسينات على فكرة النظام بسهولة بين الدورات استجابةً لأي متطلبات جديدة أو تغييرات في الأولويات. على عكس نموذج الشلال (Waterfall) التقليدي الذي يتطلب تحديد جميع المتطلبات مسبقاً، أتاحت لنا المنهجية الرشيقية تعديل مسار المشروع بناءً على المعرفة المكتسبة.

2. **تقليل المخاطر:** بدلاً من انتظار اكتمال المشروع بالكامل لاكتشاف المشاكل، يساعد هذا النموذج في الكشف المبكر عن التحديات التقنية أو المفاهيمية في مراحل مبكرة. على سبيل المثال، إذا تبين أن إحدى الميزات المقترحة معقدة تقنياً أكثر من المتوقع، يمكن معالجة هذه المشكلة في دورة تطوير مبكرة بدلاً من أن تصبح عقبة كبيرة في نهاية المشروع.

3. **الحصول على تقييم للعمل بشكل أسرع** في نهاية كل دورة تطوير، يتم تقديم جزء وظيفي من التطبيق (Increment) قابل للاستخدام. أتاح لنا ذلك عرض التقدم المحرز بشكل دوري على المشرفين للحصول على ملاحظات قيمة وملموسة، مما ضمن أن المنتج النهائي يتطور في الاتجاه الصحيح ويلبي التوقعات بدقة.

3.5 منهجية تحويل الدراسات إلى نظام

بعد اكتمال الدراستين النظرية والمرجعية، تم اتباع خطوات منهجية واضحة لتحويل المعرفة المكتسبة إلى نظام فعال. لم تكن هذه العملية عشوائية، بل كانت عملية منظمة تضمن أن كل ميزة في التطبيق لها أساس منطقي ومدرّس.

1.3.5 استخلاص المتطلبات من الأبحاث

كانت الخطوة الأولى هي تحويل الرؤى النظرية وتحليلات السوق إلى قائمة متطلبات ملموسة.

- **من الدراسة النظرية:** تم استخلاص التحديات الأساسية التي تواجه مستخدمي تطبيقات اللياقة المنزلية. على سبيل المثال، أشارت الدراسة إلى أن "الملل" و "فقدان الدافع" هما من أكبر العوائق. تم ترجمة هذه المعلومة مباشرة إلى مطلب وظيفي: "يجب أن يسمح النظام للمستخدم بإنشاء وتعديل قوائم التمارين الخاصة به بحرية"، وذلك لتلبية الحاجة النفسية للاستقلالية (Autonomy) التي تم تحديدها كنظرية أساسية لزيادة الدافع.
- **من الدراسة المرجعية:** تم تحليل ميزات التطبيقات المنافسة (مثل Nike Training Club) لتحديد الفجوات في السوق. لوحظ أن NTC يقدم محتوى مجانياً هائلاً ولكنه يفتقر إلى التخصيص. هذه الملاحظة عززت من أهمية مطلب "إنشاء قوائم تمارين مخصصة" كوسيلة لتمييز VexaFit وتقديم قيمة فريدة للمستخدم الذي يبحث عن التحكم في رحلته الرياضية.

2.3.5 نمذجة وظائف النظام

بعد تجميع قائمة واضحة بالمتطلبات، تم الانتقال إلى مرحلة النمذجة لتحديد كيفية تفاعل المستخدم مع هذه الوظائف بشكل دقيق.

- مخططات حالات الاستخدام (Use Case Diagrams): تم استخدام هذه المخططات لإنشاء خريطة مرئية وعالية المستوى لجميع الوظائف المتاحة في النظام، مع توضيح العلاقة بين المستخدم (Actor) وكل وظيفة. ساعدت هذه المخططات في تنظيم المتطلبات وتحديد نطاق العمل لكل دورة تطوير.
- السرد النصي (Textual Narrative): تم تفصيل كل حالة استخدام رئيسية من خلال السرد النصي. هذه الوثائق كانت بمثابة جسر حاسم بين المتطلبات المجردة والمهام الفنية للمطورين. فبدلاً من القول "يجب أن يتمكن المستخدم من إضافة تمرين"، يوضح السرد النصي بالتفصيل الخطوات الدقيقة: "1. المستخدم يختار قائمة التمارين. 2. النظام يعرض مكتبة التمارين. 3. المستخدم يختار تمريناً. 4. النظام يطلب تفاصيل (الجولات، التكرارات)..." وهكذا.

3.3.5 الفلسفة المعمارية: البنية النظيفة (Clean Architecture)

لتحقيق المتطلبات غير الوظيفية مثل قابلية الصيانة والتوسع، تم اعتماد البنية النظيفة (Clean Architecture) كفلسفة معمارية أساسية. هذا النمط المعماري يفرض فصلاً صارماً بين الاهتمامات (Separation of Concerns) عبر تقسيم النظام إلى طبقات منطقية مستقلة.

المبدأ الجوهرى في هذه المعمارية هو قاعدة التبعية (The Dependency Rule)، التي تنص على أن جميع الاعتماديات يجب أن تتجه نحو الداخل فقط. الطبقات الخارجية التي تحتوي على تفاصيل التنفيذ (مثل واجهات المستخدم، قواعد البيانات، وأطر العمل) تعتمد على الطبقات الداخلية التي تحتوي على منطق العمل المجرد، وليس العكس. هذا يضمن أن منطق العمل الأساسي لا يتأثر بالتغييرات في التفاصيل التقنية.

4.3.5 الاختبارات

لضمان موثوقية النظام وجودته، تم اعتماد استراتيجية اختبار متعددة المستويات:

- اختبار الوحدات (Unit Test): تم اختبار أصغر أجزاء الرماز المصدري في الواجهة الخلفية. الهدف من ذلك هو التحقق من أن كل مكون من مكونات منطق العمل يعمل بشكل صحيح وبشكل مستقل قبل دمجهم مع المكونات الأخرى، مما يساعد في اكتشاف الأخطاء في مراحل مبكرة.
- اختبار التكامل (Integration Test): تم اختبار دمج أجزاء النظام مع بعضها بعد اختبار كل جزء على حدى لضمان تكامل أجزاء النظام وتمثيل منطق العمل بشكل صحيح.
- اختبار الضغط (Stress Test): لقياس أداء النظام واستقراره تحت ظروف استخدام قاسية. تم تصميم سيناريوهات اختبار تحاكي وجود عدد كبير من المستخدمين المتزامنين لتقييم قدرة الواجهة الخلفية على التعامل مع الأحمال العالية وتحديد أية اختناقات (Bottlenecks) محتملة في الأداء.

الفصل السادس

تصميم النظام

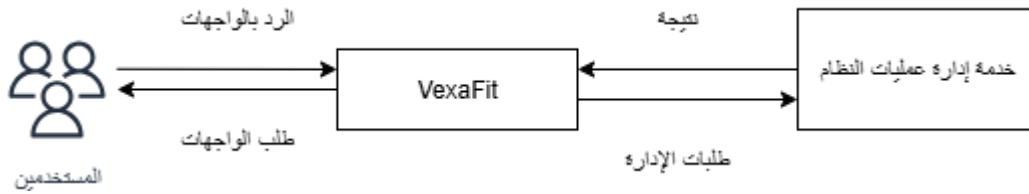
يُعرض هذا الفصل القرارات التصميمية التي بني من خلالها النظام.

1.6 مقدمة

بعد تحديد المتطلبات الوظيفية وغير الوظيفية ووضع أسلوب العمل للمشروع، تأتي مرحلة تصميم النظام التي تعد بمثابة المخطط الهندسي الذي سيتم بناء التطبيق بالاعتماد عليه. يهدف هذا الفصل إلى تقديم شرح مفصل للقرارات التصميمية التي تم اتخاذها في مشروع VexaFit، بدءًا من البنية المعمارية الكلية للنظام، مرورًا بتصميم قواعد البيانات، وانتهاءً بتصميم الواجهات وآلية عمل المكونات المختلفة معًا.

تم اتخاذ كل قرار تصميمي بناءً على المبادئ التي تم تحديدها في الدراسات السابقة، مع التركيز على تحقيق المتطلبات غير الوظيفية مثل قابلية الصيانة، التوسع، الأداء العالي، والأمان. سيستعرض هذا الفصل كيف تم تحويل المتطلبات النظرية إلى بنية تقنية متكاملة وفعالة.

2.6 مخطط النظام التصميمي



الصورة (12): مخطط النظام التصميمي

3.6 البنية المعمارية للنظام (System Architecture)

1.3.6 بنية الواجهة الخلفية (.NET)

تم تقسيم الواجهة الخلفية إلى أربع طبقات رئيسية:

1. **طبقة المجال (Domain Layer):** هي قلب النظام وتحتوي على الكيانات (Entities) التي تمثل المفاهيم الأساسية في المشروع (مثل Workout, Exercise) وقواعد العمل التي لا تتغير بتغير التكنولوجيا المستخدمة حيث تم تجزئها كاملاً بلغة C# بمعزل عن إطار العمل المستخدم في الطبقات الأعلى.
2. **طبقة التطبيق (Application Layer):** تحتوي على منطق العمل الخاص بالتطبيق (Use Cases). هي التي تتسق التفاعل بين طبقة المجال والطبقات الخارجية، وتعرف الواجهات (Interfaces) التي يجب على طبقة البنية التحتية تنفيذها.
3. **طبقة البنية التحتية (Infrastructure Layer):** تحتوي على التنفيذ الفعلي للواجهات المعرفة في طبقة التطبيق. تشمل هذه الطبقة كل ما يتعلق بالعوامل الخارجية مثل التعامل مع قاعدة البيانات باستخدام Entity Framework Core، أو الاتصال بخدمات خارجية.
4. **طبقة واجهة برمجة التطبيقات (API Layer):** هي نقطة الدخول إلى النظام. تحتوي على وحدات التحكم (Controllers) التي تستقبل طلبات HTTP من العملاء (تطبيق الموبايل أو لوحة التحكم) وتمررها إلى طبقة التطبيق، ثم تعيد الاستجابة المناسبة.

تم تضمين عدة مفاهيم بشكل رئيسي والتي ظهرت من خلال تقسيم معمارية النظام وكيفية تجزئ كل طبقة:

1. **التمحور حول المجال:** يكمن جوهر التصميم والغاية من هذه المعماري بأن تكون الواجهة الخلفية تضمن توجه التمحور حول المجال (Domain Centric) مما يعني أن طبقة المجال ليست مجرد طبقة أخرى في النظام، بل هي مركزه الأساسي. أيضاً تم عزل منطق العمل الحقيقي (Business Logic) حيث تحتوي طبقة المجال على الكيانات (Entities) وقواعد العمل الجوهرية (Enterprise Business Rules) التي تمثل المفاهيم الأساسية للمشروع. هذه القواعد هي الأكثر استقراراً والأقل تغييراً مع مرور الزمن.
2. **استقلالية تامة عن إطار العمل:** تم تطوير طبقة المجال بلغة C# فقط دون التطرق إلى إطار عمل أو الاعتماد عليه مثل Asp.Net Core أو أية مكتبة للتعامل مع قاعدة المعطيات مثل Entity Framework وهذه الاستقلالية تضمن أن منطق العمل الأساسي مستمر حتى بتغير إطار العمل مما يعطي النظام مرونة.

3. تدفق التبعية من الخارج للداخل: المبدأ الحاكم في هذه البنية هو تدفق التبعية من الخارج للداخل (The Dependency Rule)، والذي يفرض أن تتجه الاعتماديات بشكل نحو مركز النظام. في هذا المشروع، تعتمد طبقة واجهة برمجة التطبيقات (API Layer) على الواجهات التي تعرفها طبقة التطبيق (Application Layer)، كما تعتمد طبقة البنية التحتية (Infrastructure Layer) على نفس الواجهات لتقوم بتنفيذها، بينما تعتمد طبقة التطبيق بدورها على طبقة المجال (Domain Layer). وتبقى طبقة المجال، باعتبارها قلب النظام، مستقلة تمامًا ولا تعتمد على أي طبقة أخرى. هذا التدفق أحادي الاتجاه يضمن عزل منطق العمل وحمايته من أي تغييرات في التفاصيل التقنية الخارجية كقاعدة البيانات أو أطر العمل، مما يعزز استقرار النظام وقابليته للصيانة.

4. توحيد الخدمات الأساسية: على الرغم من العزل بين الخدمات الأساسية Services تم بناء واجهة عامة IService والتي تصرح عن العمليات الأساسية التي يمكن تطبيقها على أي كيان (Create, Read, Update, Delete – CRUD) ما يفرض معيارًا موحدًا، وتم تجزئة هذه الواجهة لكل خدمة بما يتناسب مع منطق العمل الذي يتعلق بالكيان Entity

5. الفصل بين الخدمات: كل خدمة متخصصة في طبقة البنية التحتية مستقلة عن باقي الخدمات، فمثلاً خدمة قوائم التمارين WorkoutService تقوم بتجزئة الواجهة IService بحسب منطق العمل الخاص بها دون التعاون مع باقي الخدمات من مثل ExerciseService حيث لا تتصل الخدمات ببعضها.

2.3.6 بنية الواجهة الأمامية (Flutter)

تم تطبيق نفس المبادئ على تطبيق الموبايل لضمان فصل منطق العرض عن منطق العمل وتحقيق تجربة مستخدم متناسقة:

1. طبقة البيانات (Data Layer): تحتوي على نماذج البيانات (Models) التي تعكس شكل البيانات القادمة من الـ API، وتعرف واجهات المستودعات (Repository Interfaces) التي تحدد العمليات المتاحة على هذه البيانات.
2. طبقة البنية التحتية (Infrastructure Layer): تقوم بتنفيذ واجهات المستودعات. هي المسؤولة عن الاتصال بالـ API لجلب البيانات وإرسالها، وتحويل البيانات من صيغة JSON إلى نماذج Dart.
3. طبقة العرض (Presentation Layer): تشمل كل ما يتعلق بواجهة المستخدم (UI)، وتنقسم إلى:
 - Screens/Widgets: المكونات المرئية التي يراها المستخدم.
 - ViewModels/Providers: الطبقة المسؤولة عن إدارة حالة كل شاشة (State Management)، حيث تستدعي المستودعات لجلب البيانات وتجهيزها للعرض، وتستجيب لتفاعلات المستخدم.
1. الطبقة الأساسية (Core Layer): تعمل كمركز للإعدادات والأدوات المشتركة التي تخدم التطبيق بأكمله. تحتوي على:

- URLs: إدارة مركزية لجميع روابط الـ API المستخدمة في التطبيق.

- **Themes**: تعريف السمات (ThemeData) لتوحيد الألوان والخطوط والمظهر العام للتطبيق.
- **Utils**: مجلد يحتوي على أدوات خدمية مثل إدارة وتخزين (JWT Token)، بالإضافة إلى تعريف الثوابت والمتغيرات المتعددة (Enums).
- **Dio Client**: إعداد وتهيئة نسخة من مكتبة Dio للتواصل مع الخادم المحلي (Local Server) وإجراء طلبات الشبكة.

إن تنظيم بنية المجلدات والملفات لا يقل أهمية عن اختيار النمط المعماري، فهو التطبيق العملي لمبدأ فصل الاهتمامات على مستوى الملفات، مما ينعكس بشكل مباشر على إنتاجية فريق العمل وسهولة صيانة المشروع على المدى الطويل. تم تصميم الهيكلية لتحقيق أهداف محددة تتمحور حول الوضوح، قابلية إعادة الاستخدام، والتناسق.

1. **مركزية المكونات القابلة لإعادة الاستخدام (Reusable Widgets)** : يكمن الهدف من إنشاء مجلد widgets مخصص في تطبيق مبدأ "لا تكرر نفسك (Don't Repeat Yourself – DRY)" بدلاً من إعادة كتابة شيفرة المكونات المرئية الشائعة في كل شاشة، يتم بناؤها مرة واحدة في هذا المجلد واستدعاؤها عند الحاجة. هذا النهج يقدم ثلاث فوائد رئيسية:

- **ضمان التناسق البصري** : جميع الأزرار، حقول الإدخال، وبطاقات العرض ستبدو وتعمل بنفس الطريقة تماماً في كل أنحاء التطبيق، مما يعزز هوية بصرية موحدة.
- **تسريع عملية التطوير** : يمكن للمطورين بناء شاشات جديدة بسرعة أكبر عبر تجميع هذه المكونات الجاهزة معاً.
- **سهولة التعديل** : إذا استدعى الأمر تغيير تصميم زر معين، يتم تعديله في ملف واحد فقط، وينعكس هذا التغيير تلقائياً في كل مكان تم استخدامه فيه.

2. **التنظيم المنطقي للشاشات (Logical Screen Organization)** : إن فصل الشاشات إلى مجلدين main_app_screens و work_flow_screens يخدم غرضاً هيكلياً بالغ الأهمية يتجاوز مجرد الترتيب.

- **وضوح بنية التطبيق**: يسهل هذا التقسيم على أي مطور (جديد أو قديم في المشروع) فهم الهيكل العام للتطبيق بسرعة. "الشاشات الرئيسية" تمثل الواجهات النهائية المستقرة، بينما "شاشات مسار العمل" تمثل مسارات مؤقتة لإنجاز مهمة.
- **عزل مسارات المستخدم (User Experience)**: كل مسار عمل، مثل إنشاء حساب، يصبح وحدة منطقية معزولة في مجلد خاص به. هذا يجعل عملية تتبع الأخطاء أو إضافة ميزات جديدة ضمن هذا المسار أكثر سهولة وتركيزاً، دون التأثير على باقي أجزاء التطبيق.
- **تحسين قابلية الصيانة**: عندما تكون الملفات منظمة بناءً على وظيفتها، يصبح العثور على الرماز المصدري المسؤول عن ميزة معينة أمراً سريعاً وبديهيًا، مما يقلل من الوقت اللازم للصيانة والتطوير.

3. توحيد هوية التطبيق عبر السمات: (Theming for Brand Consistency) يُعد استخدام ملف ThemeData مركزي مهما في تحقيق تناسق بصري. من خلال تعريف لوحة الألوان المعتمدة، أحجام وأنواع الخطوط، وأشكال المكونات في مكان واحد، يتم فرض هوية بصرية موحدة على التطبيق بأكمله. الفائدة هنا لا تقتصر على المظهر الجمالي، بل تمتد إلى الكفاءة؛ حيث إن أي قرار مستقبلي بتغيير الألوان يمكن تنفيذه عبر تعديل ملف واحد، مما يضمن تطبيق التغييرات بشكل فوري وشامل على جميع الشاشات دون استثناء.

الفصل السابع

الأدوات المستخدمة

يعرض هذا الفصل الأدوات المستخدمة لتنفيذ هذا المشروع.

1.7 ASP.NET Core

إطار عمل مفتوح المصدر وعالي الأداء تم تطويره بواسطة مايكروسوفت لبناء تطبيقات الويب الحديثة وواجهات برمجة التطبيقات (APIs). تم اختياره لبناء الواجهة الخلفية لـ VexaFit للأسباب التالية:

- **الأداء العالي:** يُعرف ASP.NET Core بسرعته وكفاءته، مما يجعله مناسباً للتطبيقات التي تتطلب التعامل مع عدد كبير من الطلبات المتزامنة، وهو متطلب أساسي لتطبيق لياقة بدنية.
- **متعدد المنصات:** يمكن تشغيل تطبيقات ASP.NET Core على أنظمة التشغيل Windows, macOS و Linux، مما يوفر مرونة كبيرة في خيارات الاستضافة والنشر.
- **نظام بيئي متكامل:** يوفر الإطار أدوات مدمجة قوية للأمان، والمصادقة (Authentication)، والترخيص (Authorization)، وإدارة الإعدادات، مما يسرع من عملية التطوير.

2.7 Entity Framework Core

إطار عمل للوصول إلى البيانات (Object-Relational Mapper - O/RM) خاص بـ .NET. تم استخدامه كوسيط بين شيفرة التطبيق وقاعدة البيانات.

- **تبسيط التعامل مع قاعدة البيانات:** يسمح EF Core للمطورين بالتعامل مع قاعدة البيانات باستخدام كائنات #C بدلاً من كتابة استعلامات SQL بشكل مباشر، مما يقلل من الأخطاء ويزيد من الإنتاجية.
- **نهج Code-First:** تم استخدامه لتعريف نماذج البيانات في الكود أولاً، ثم يقوم EF Core بإنشاء مخطط قاعدة البيانات تلقائياً، مما يسهل إدارة التغييرات على بنية قاعدة البيانات من خلال Migrations.

PostgreSQL 3.7

نظام إدارة قواعد بيانات علائقية (RDBMS) قوي ومفتوح المصدر. تم اختياره كقاعدة بيانات أساسية للمشروع بسبب، الموثوقية والاستقرار حيث يشتهر PostgreSQL بكونه نظامًا موثوقًا ومستقرًا وقادرًا على التعامل مع كميات كبيرة من البيانات والمعاملات المعقدة، ويتوافق بشكل ممتاز مع Entity Framework Core.

Json Web Token (JWT) 4.7

الرمز (JWT) هو غرض JSON يُستخدم لنقل المعلومات بين طرفين بشكل آمن عبر الويب، يتم استخدام JWT على نطاق واسع في التطبيقات الحديثة كآلية مصادقة عديمة الحالة. بعكس cookies رموز JWT يمكن استخدامها للمصادقة في تطبيقات الويب، تطبيقات الجوال وتطبيقات سطح المكتب (Desktop application).

Flutter 5.7

إطار عمل لتطوير واجهات المستخدم (UI Toolkit) تم تطويره بواسطة جوجل لبناء تطبيقات جميلة ومترجمة أصلاً (natively compiled) للهواتف المحمولة والويب و سطح المكتب من قاعدة كود واحدة. تم استخدامه لبناء تطبيق VexaFit للمستخدمين.

- **قاعدة كود واحدة:** الميزة الأبرز هي القدرة على كتابة الكود مرة واحدة ونشره على نظامي iOS و Android، مما يوفر الكثير من الوقت والجهد.
- **تطبيق التغييرات بسرعة:** ميزة "Hot Reload" تسمح للمطورين برؤية التغييرات التي يجرونها على الكود بشكل فوري تقريبًا، مما يسرع بشكل كبير من عملية بناء وتجربة الواجهات.

React 6.7

مكتبة JavaScript لبناء واجهات المستخدم تم تطويرها بواسطة فيسبوك. تم استخدامها لبناء لوحة تحكم المسؤول (Admin Dashboard) كتطبيق ويب أحادي الصفحة (SPA).

- **بنية قائمة على المكونات (Component-Based):** تسمح ببناء واجهات مستخدم معقدة عن طريق تقسيمها إلى مكونات صغيرة ومستقلة وقابلة لإعادة الاستخدام.
- **كفاءة الأداء:** تستخدم React ما يسمى بـ Virtual DOM لتحسين أداء عمليات تحديث الواجهة، مما يجعل التطبيق سريع الاستجابة.

xUnit 7.7

إطار عمل مفتوح المصدر لاختبار الوحدات (Unit Testing) في تطبيقات .NET. يعتبر من أكثر الأطر استخدامًا إلى جانب MSTest وNUnit، ويدعمه فريق .NET الرسمي، ويُستخدم على نطاق واسع في مشاريع ASP.NET Core و .NET بشكل عام.

- يدعم بيئة .NET Core بشكل مدمج.
- يستخدم صفات حديثة مثل [Fact] للاختبارات العادية، و [Theory] للاختبارات بمعطيات مختلفة.
- يوفر آليات مرنة لتهيئة واستخدام البيانات المشتركة عبر ClassFixture أو CollectionFixture.
- يسمح بكتابة اختبارات قابلة للتوسعة والتنظيم بسهولة.

Moq 8.7

إطار عمل خاص بإنشاء الكائنات الوهمية (Mock Objects) ويُستخدم مع اختبارات الوحدات في .NET. يسمح Moq بعزل الوحدة المراد اختبارها عن التبعية الخارجية مثل قواعد البيانات أو الخدمات الخارجية، مما يساعد على اختبار منطق الوحدة فقط.

- إنشاء كائنات وهمية من الواجهات أو الكائنات المجردة.
- تحديد قيم إرجاع مخصصة عند استدعاء دوال معينة.
- التحقق مما إذا تم استدعاء الدوال، وعدد مرات الاستدعاء، والمعاملات التي تم تمريرها إليها.

Grafana k6 9.7

إطار عمل مفتوح المصدر لاختبار الأداء (Performance Testing) واختبار التحمل (Load Testing) يُستخدم لقياس قدرة الأنظمة والخوادم على التعامل مع أعداد كبيرة من المستخدمين أو الطلبات في وقت قصير.

Matplotlib 10.7

مكتبة مفتوحة المصدر بلغة **Python** تُستخدم لرسم الرسوم البيانية والتصوير البياني للبيانات. تُعد من أشهر المكتبات في هذا المجال، وتُستخدم في مجالات متعددة مثل تحليل البيانات، الذكاء الاصطناعي، الإحصاء، واختبارات الأداء مثل نتائج أداة k6.

Git 11.7

نظام إدارة إصدارات موزّع (Distributed Version Control System)، يُستخدم لمتابعة التعديلات على الشيفرة المصدرية أثناء عملية التطوير و يتيح للمطورين:

- حفظ تاريخ التعديلات والرجوع إلى أي إصدار سابق من المشروع.
- العمل على ميزات أو إصلاحات مختلفة باستخدام الفروع (Branches) .
- اكتشاف التغييرات ودمجها بطريقة آمنة ومنظمة.

Github 12.7

GitHub هو منصة سحابية مبنية على **Git** ، تُستخدم لاستضافة المشاريع ومشاركتها وإدارتها عبر الإنترنت، كما توفر العديد من الميزات الإضافية، منها:

- استضافة المستودعات (Repositories) ومشاركة الرماز المصدري مع الآخري.
- دعم التعاون من خلال طلبات السحب (Pull Requests) ومراجعة الرماز المصدري.

go_router 13.7

حزمة إدارة التنقل الحديثة المعتمدة من فريق Flutter ، تتيح تعريف المسارات (Routes) بطريقة منظمة وسهلة القراءة، وتدعم حالات التنقل المعقدة مثل التوجيه بناءً على الحالة أو التوجيه المتداخل. تم استخدامها لتسهيل التنقل بين شاشات التطبيق بطريقة مرنة، مثل التوجيه بعد تسجيل الدخول أو إعادة التوجيه في حالة انتهاء صلاحية الجلسة.

dio 14.7

مكتبة HTTP قوية ومرنة تتيح إرسال واستقبال الطلبات الشبكية بسهولة مع دعم كامل للتعامل مع الاستثناءات، وإعادة المحاولة، والمصادقة، وتحميل الملفات، واعتراض الطلبات. تم استخدامها في المشروع لإرسال واستقبال البيانات من وإلى الخادم (API)، حيث تُستخدم لإجراء عمليات مثل تسجيل الدخول، استرجاع التمارين، وإرسال البيانات إلى الخادم بطريقة آمنة ومنظمة.

provider 15.7

مكتبة لإدارة الحالة (State Management) تُعد من أبسط وأكثر الطرق فعالية في تطبيقات Flutter لفصل منطق العمل عن واجهة المستخدم. تم استخدامها لإدارة البيانات المشتركة بين الشاشات، مثل حالة تسجيل الدخول، تفاصيل المستخدم، أو قائمة التمارين الخاصة، مما ساهم في تنظيم الكود وتقليل الاعتماد المباشر بين المكونات.

flutter_secure_storage 16.7

مكتبة توفر واجهة آمنة لتخزين البيانات الحساسة مثل رموز المصادقة (Tokens) على الأجهزة المحمولة، باستخدام التشفير الأساسي لنظام التشغيل. تم استخدامها لتخزين رمز الوصول (JWT Access Token) بطريقة آمنة بعد تسجيل الدخول، بهدف الحفاظ على سرية البيانات ومنع الوصول غير المصرح به.

الفصل الثامن

تنجيز النظام

يُعرض هذا الفصل كيفية تنجيز النظام مع تفصيل كل جزء من أجزائه.

1.8 مقدمة

يعرض هذا الفصل كيفية تنجيز نظام VexaFit وتحويل التصميم المعمارية والمخططات التي تم وضعها في الفصول السابقة إلى مكونات برمجية فعالة ومتكاملة. خلال عملية التطوير، تم رفع الرماز المصدري (Source Code) الخاص بكل جزء من أجزاء النظام (الواجهة الخلفية، الواجهة الأمامية، ولوحة التحكم) باستخدام أداة **Git** إلى مستودعات (Repositories) منفصلة ضمن منصة **GitHub**. تم اتباع استراتيجية الفرع (Branching Strategy) حيث يتم تطوير الميزات الجديدة في فروع خاصة بها، ثم دمجها لاحقاً في الفرع الرئيسي، مما يضمن بيئة عمل منظمة وتعاونية.

يحقق هذا الأسلوب مبدأ التكامل المستمر (Continuous Integration)، حيث يتم تجميع الرماز البرمجي ضمن مستودع مركزي، مما يسهل على فريق العمل التعاون واختبار التغييرات بشكل دوري.

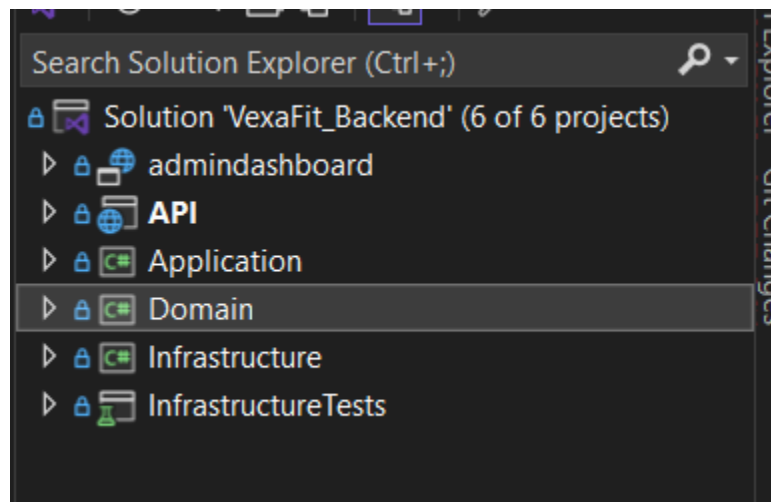
2.8 تنجيز الواجهة الخلفية (BackEnd)

تم بناء الواجهة الخلفية باستخدام إطار العمل **ASP.NET Core** و لغة البرمجة **C#**. تم اختيار هذه التقنيات لقدرتها على توفير أداء عالٍ، وقابلية للتطوير، ونظام بيئي متكامل يدعم بناء تطبيقات قوية وأمنة. تم اتباع معمارية **Clean Architecture** بدقة لضمان فصل الاهتمامات وجعل النظام قابلاً للصيانة والاختبار.

يتألف نظام الواجهة الخلفية من أربع طبقات رئيسية، حيث تتدفق الطلبات من الطبقات الخارجية إلى الداخلية:

- **API Layer**: هي الطبقة المسؤولة عن استقبال طلبات الـ HTTP. تحتوي على وحدات التحكم (Controllers) التي تمثل نقاط النهاية (Endpoints) للنظام. تقوم هذه الطبقة بالتحقق من صحة الطلبات الواردة ثم تمريرها إلى طبقة التطبيق.
- **Application Layer**: المنسق و Orchestrator لمنطق العمل Business Logic.
- **Infrastructure Layer**: تحتوي على التنفيذ الفعلي للواجهات المعرفة في طبقة التطبيق. تم استخدام **Entity Framework Core** في هذه الطبقة للتواصل مع قاعدة البيانات **PostgreSQL** وتنفيذ عمليات القراءة والكتابة.

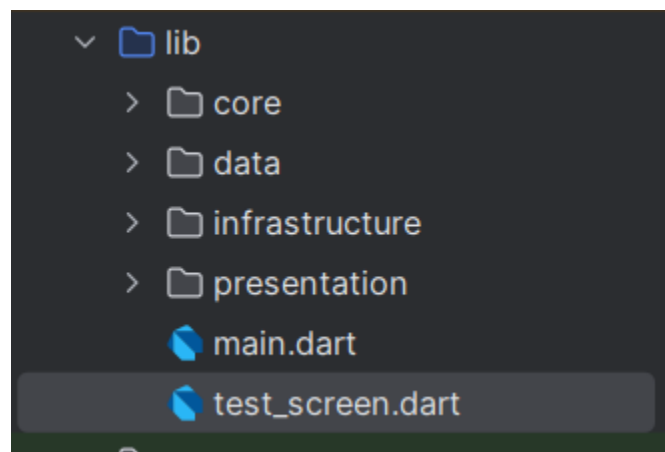
- **Domain Layer**: قلب النظام وتحتوي على الكيانات (Entities) التي تمثل البيانات الأساسية (مثل المستخدم، التمرين، إلخ)، وهي منجزة فقط بلغة C# ومستقلة عن أطر العمل في الطبقات الأعلى.



الصورة (13): توضيح البنية لتنفيذ الواجهة الخلفية

3.8 تنفيذ الواجهة الأمامية (تطبيق الموبايل)

تم بناء تطبيق الموبايل باستخدام إطار العمل **Flutter** ولغة البرمجة **Dart**. تم اختيار Flutter لقدرته على بناء تطبيقات تعمل على نظامي iOS و Android من قاعدة كود واحدة، مما يضمن تجربة مستخدم متسقة ويسرع من عملية التطوير.

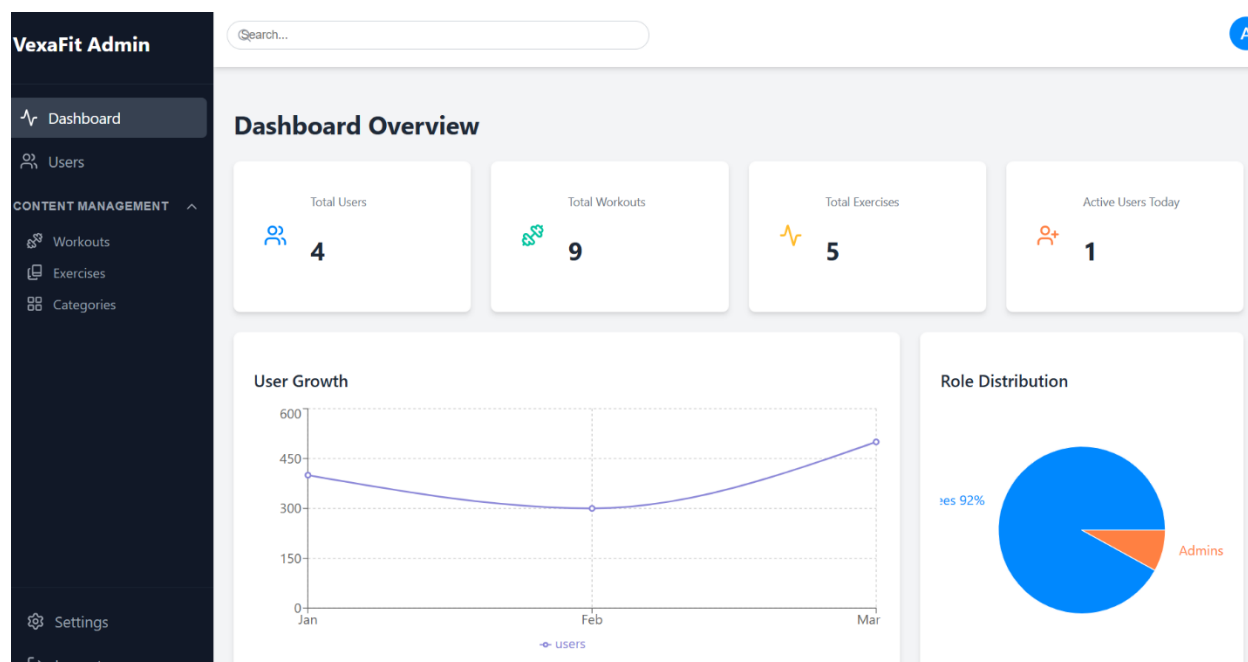


الصورة (14): توضيح البنية لتنفيذ الواجهة الأمامية للمستخدم

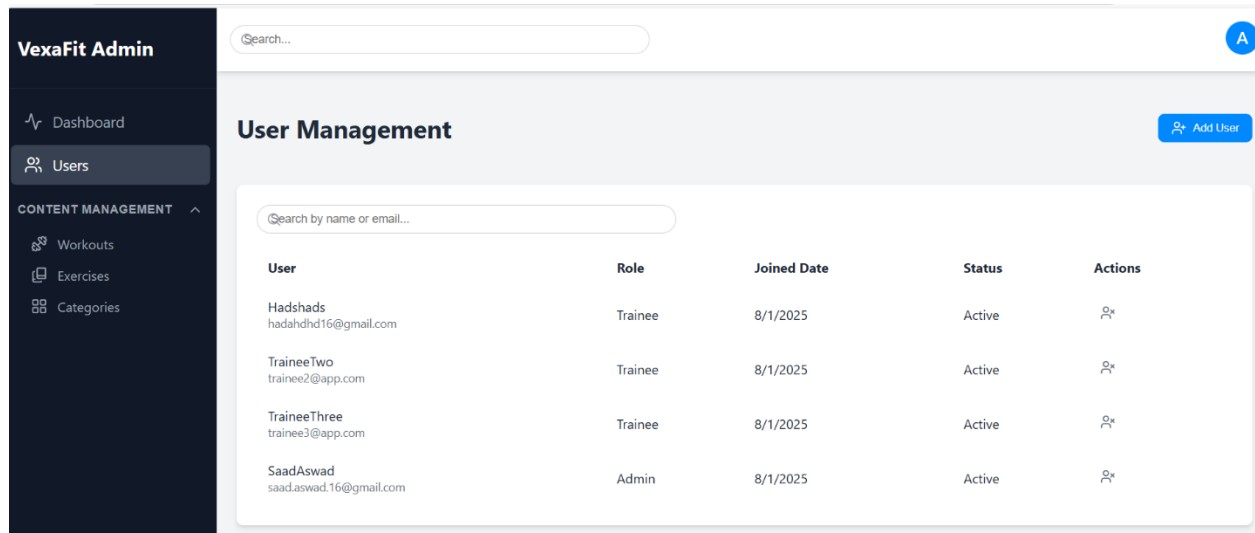
4.8 تنجيز لوحة تحكم المستخدم

تم بناء لوحة تحكم المسؤول كتطبيق ويب أحادي الصفحة (Single-Page Application) باستخدام مكتبة **React**. اختيار **React** لقدرتها على بناء واجهات مستخدم تفاعلية وسريعة الاستجابة، ولسهولة إدارتها بفضل بنيتها القائمة على المكونات (Component-Based). . تتيح لوحة تحكم المسؤول إمكانية التحكم بشكل كامل بقاعدة المعطيات وإلغاء أو تفعيل حساب أحد المستخدمين، كما تتيح عرض الإحصاءات اليومية والشهرية.

- **بنية المكونات:** تم تقسيم واجهة المستخدم إلى مكونات صغيرة وقابلة لإعادة الاستخدام (مثل الجداول، الأزرار، النماذج). كل صفحة في لوحة التحكم (مثل صفحة إدارة المستخدمين) هي عبارة عن مكون رئيسي يتألف من عدة مكونات فرعية.
- **الاتصال بالـ API:** تم استخدام مكتبة **Axios** لإجراء طلبات HTTP إلى الواجهة الخلفية لجلب البيانات (مثل قائمة المستخدمين) وإرسال التحديثات (مثل تعديل حالة مستخدم).
- **إدارة الحالة:** تم الاعتماد على **React Hooks** مثل **useState** و **useEffect** لإدارة حالة المكونات، مثل تخزين قائمة المستخدمين التي تم جلبها من الـ API أو تتبع حالة التحميل.

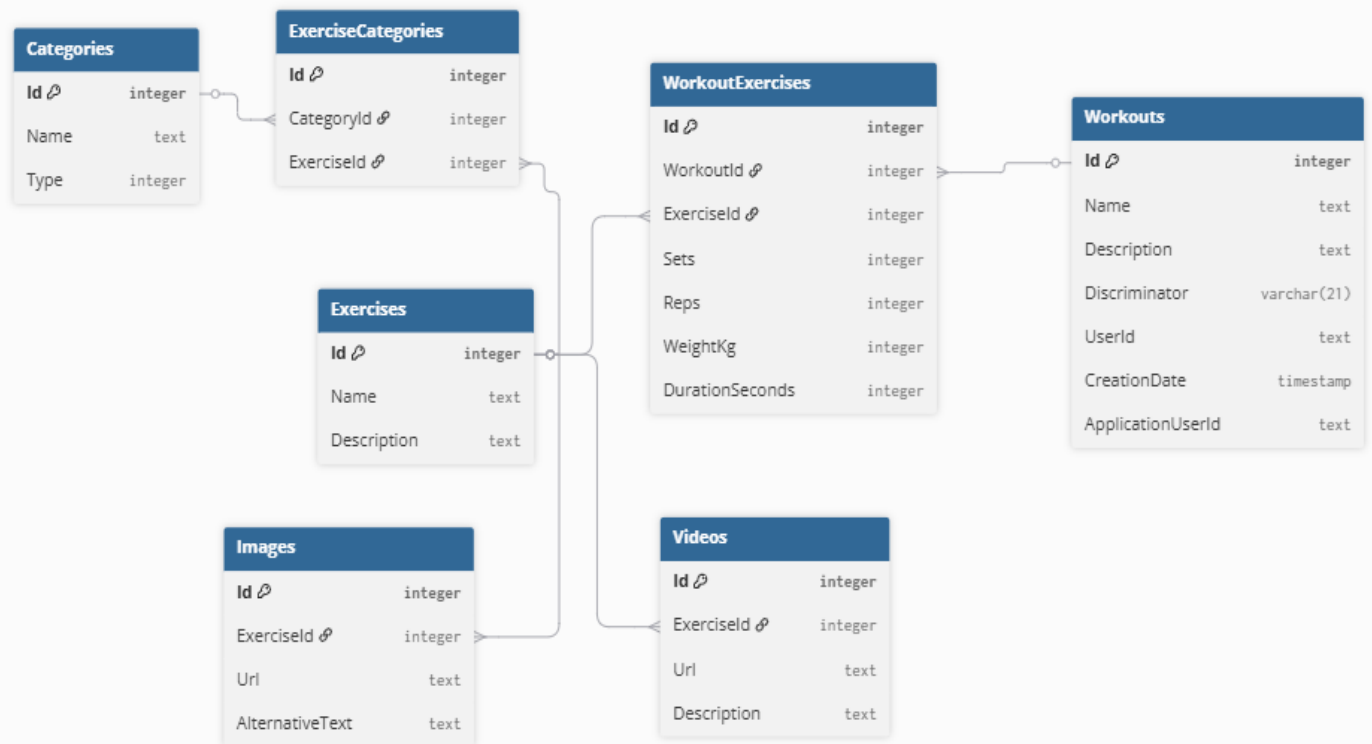


الصورة (15): الصفحة الرئيسية لواجهة المسؤول



الصورة (16): صفحة إدارة المستخدمين لواجهة المسؤول

5.8 تصميم قاعدة المعطيات



الصورة (17): تصميم قاعدة معطيات خدمة إدارة عمليات النظام

6.8 مخطط النظام التنفيذي



الصورة (18): كيفية التواصل بين الخدمات ضمن النظام مع الأدوات المستخدمة

الفصل التاسع

اختبارات النظام ومناقشة النتائج

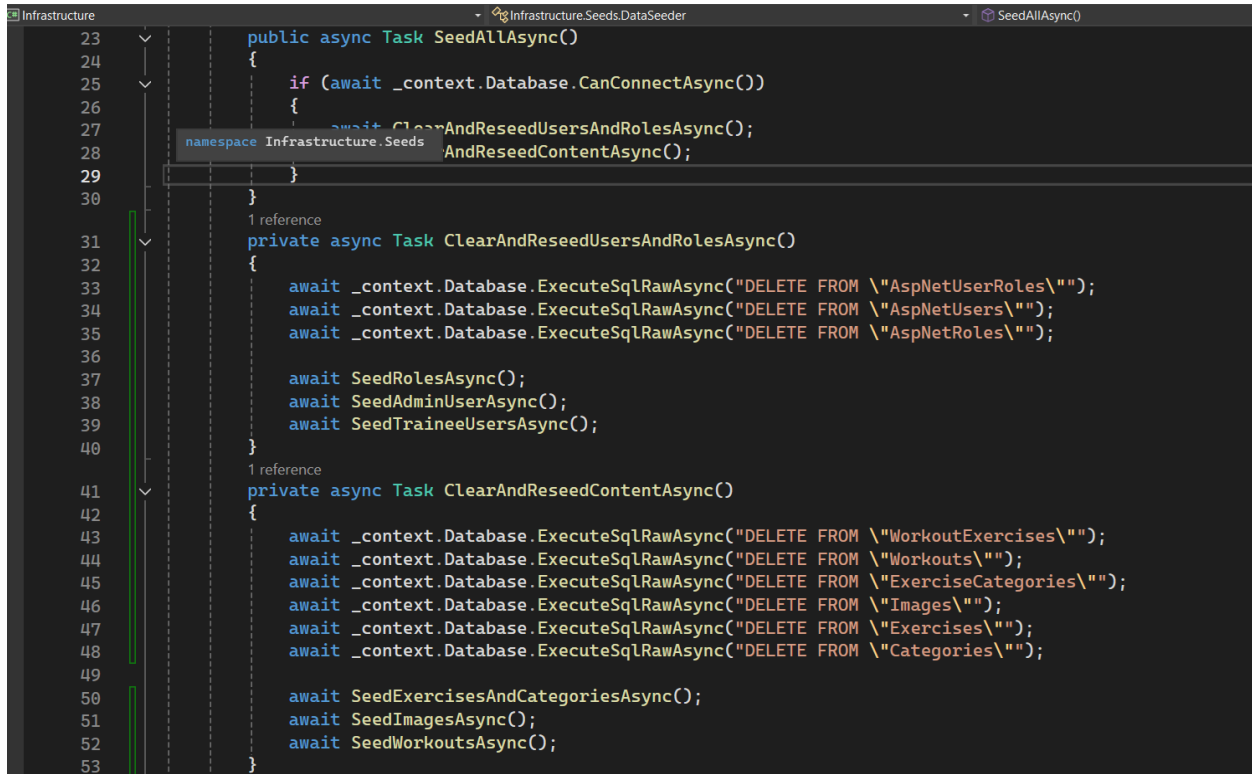
يوضح هذا الفصل الاختبارات التي تم تطبيقها للتأكد من تلبية النظام للمتطلبات.

1.9 اختبار خدمة إدارة عمليات النظام

من أجل اختبار قدرة الواجهة الخلفية على الاستجابة لعدد كبير من الطلبات بشكل مفاجئ ومستمر، تم إنشاء اختبارات لأداء هذه المهمة باستخدام الأداة Grafana k6، سيتم عرض كيفية تنجيز هذه الاختبارات وتفاصيلها مع نتائجها، كما تم إضافة مجموعة من التمارين وقوائم التمارين والأصناف والصور إلى قاعدة المعطيات من أجل اختبار الوحدات واختبارات التكاملية واختبارات الحماية Unit, Integration and Security Tests.

1.1.9 التجهيز للاختبارات

تم إضافة مجموعة من قوائم التمارين والتمارين والأصناف لقاعدة البيانات لأجل أداء الاختبارات كما هو موضح في الصورة 14 حيث يتم في كل مرة حذف المعطيات وإعادة تخزينها لوضع بيئة اختبار ثابتة في كل مرة.



الصورة (19): ملئ قاعدة البيانات بعلومات للاختبار

2.1.9 اختبار الوحدات واختبارات التكامل واختبارات الحماية

تم إنشاء مجموعة من اختبارات الوحدة Unit Tests واختبارات التكامل Integration Tests لاختبار مراعاة الخدمة للمتطلبات الموكلة إليها، تم إنشاء وحدات الاختبار لحالات الاستخدام الأساسية كما تم اختبار التكامل بين الخدمات واختبارات الحماية لضمان عدم وصول المستخدمين لمعلومات تتعلق بمستخدمين آخرين والتأكد من عدم تجاوز المستخدمين للصلاحيات المعطاة لهم.

WorkoutServiceTests (32)	93 ms
✓ AddToWorkout_WithNonExistentWorkout_ShouldThrow	1 ms
✓ CreateAsync_WhenCalled_AssignsCorrectUserIdAndSaves	4 ms
✓ CreateAsync_WhenDtolsValid_ShouldReturnMappedDTO	3 ms
✓ CreateAsync_WhenUserNotAuthenticated_ShouldThrow	2 ms
✓ CreateAsync_WithNullDto_ShouldThrow	< 1 ms
✓ DeleteAsync_WhenUserDoesNotOwnWorkout_ShouldThro...	1 ms
✓ DeleteAsync_WhenUserIsAdmin_ShouldDeleteAnyWorkout	1 ms
✓ DeleteAsync_WhenUserOwnsWorkout_ShouldDeleteSucces...	1 ms
✓ DeleteAsync_WhenWorkoutNotFound_ShouldThrowKeyNot...	< 1 ms
✓ DeleteAsync_WithPredefinedWorkoutAsAdmin_ShouldSucc...	1 ms
✓ DeleteBulkAsync_WithEmptyList_ShouldNotCallRepository	< 1 ms
✓ GetAllAsync_ForNonAdmin_ShouldReturnOnlyOwnedAndPr...	23 ms
✓ GetAllAsync_WhenNoWorkoutsMatch_ShouldReturnEmptyL...	4 ms
✓ GetAllAsync_WithAdminUser_ShouldReturnAllWorkouts	2 ms
✓ GetAllAsync_WithDescriptionFilter_ShouldReturnFilteredRes...	5 ms
✓ GetAllAsync_WithFullFilter_ShouldApplyAllFilters	5 ms
✓ GetAllAsync_WithNameFilter_ShouldReturnFilteredResults	4 ms
✓ GetAllAsync_WithPredefinedFilter_ShouldReturnOnlyPredefi...	4 ms
✓ GetAllAsync_WithUserIdFilterForNonAdmin_ShouldIgnoreFil...	3 ms
✓ GetByIdAsync_UnauthorizedAccessAttempt_ShouldThrowN...	2 ms
✓ GetByIdAsync_WhenWorkoutDoesNotExist_ShouldThrowEx...	2 ms
✓ GetByIdAsync_WhenWorkoutExists_ShouldReturnCorrectW...	8 ms
✓ GetByIdAsync_WhenWorkoutIsPredefined_ShouldReturnFor...	1 ms
✓ GetByIdAsync_WithInvalidId_ShouldThrowProperException	2 ms
✓ UpdateAsync_WhenDtolsValid_ShouldReturnUpdatedDTO	1 ms
✓ UpdateAsync_WhenUserDoesNotOwnWorkout_ShouldThro...	1 ms
✓ UpdateAsync_WhenUserIsAdmin_ShouldUpdateAnyWorkout	1 ms

الصورة (20): النتائج المحلية لاختبارات الوحدة ضمن خدمة إدارة العمليات

test	Duration	Tests	Error
InfrastructureTests (56)	1.8 sec		
Infrastructure.AppServices.Workout.Tests (56)	1.8 sec		
WorkoutExerciseTests (24)	1.7 sec		
AddToWorkout_WhenAdmin_IgnoresOwnership	1.4 sec		
AddToWorkout_WhenDataIsValid_ShouldInsertSuccessfully	12 ms		
AddToWorkout_WhenExerciseAlreadyExists_ShouldThrowEx...	255 ms		
AddToWorkout_WhenExerciseIsAlreadyPresent_ShouldThro...	3 ms		
AddToWorkout_WhenExerciseIsNotPresent_ShouldAddSucc...	5 ms		
AddToWorkout_WhenExerciseNotFound_ShouldThrowExce...	2 ms		
AddToWorkout_WhenExerciseNotFound_ThrowsException	2 ms		
AddToWorkout_WhenUnauthorizedUser_ThrowsException	2 ms		
AddToWorkout_WhenUserDoesNotOwnWorkout_ShouldThr...	2 ms		
AddToWorkout_WhenWorkoutNotFound_ShouldThrowExce...	2 ms		
AddToWorkout_WhenWorkoutNotFound_ThrowsException	2 ms		
DeleteFromWorkout_WhenEntryExistsAndUserIsOwner_Sho...	41 ms		
DeleteFromWorkout_WhenEntryNotFound_ThrowsException	4 ms		
DeleteFromWorkout_WhenLinkNotFound_ShouldThrowKey...	3 ms		
DeleteFromWorkout_WhenUnauthorizedUser_ThrowsExcep...	3 ms		
DeleteFromWorkout_WhenUserDoesNotOwnWorkout_Sho...	3 ms		
DeleteFromWorkout_WhenUserOwnsWorkout_ShouldDelet...	3 ms		
UpdateExerciseInWorkout_WhenAdmin_IgnoresOwnership	5 ms		
UpdateExerciseInWorkout_WhenEntryExistsAndUserIsOwne...	2 ms		
UpdateExerciseInWorkout_WhenEntryNotFound_ShouldThr...	1 ms		
UpdateExerciseInWorkout_WhenEntryNotFound_ThrowsExc...	1 ms		
UpdateExerciseInWorkout_WhenUnauthorizedUser_Throws...	1 ms		
UpdateExerciseInWorkout_WhenUserDoesNotOwnWorkout...	1 ms		
UpdateExerciseInWorkout_WhenUserOwnsWorkout_Should...	1 ms		
WorkoutServiceTests (32)	93 ms		

الصورة (21): النتائج المحلية لاختبارات التكامل والحماية ضمن خدمة إدارة العمليات

توضح الصورتان 15 و 16 نجاح اختبارات الوحدة والتكامل عند التشغيل محليا.

ملاحظة:

ساعدت اختبارات الوحدة على تحديد مشكلة في تجميع قوائم التمارين وعرضها للمستخدم، وذلك عند قصر نمط كل من قوائم التمارين المعدة مسبقا والتي أنشأها المستخدم، تم قصر النمط على الصف الأب Workout، وعند جمع المعطيات تبين

وجود خطأ. على الرغم من الاعتماد على Entity Framework لأجل جمع المعلومات من الجداول وتجميعها في استعلام واحد إلى أن الخطأ كان بسبب الـ Entity Framework.

```
var predefinedQuery = query.OfType<PredefinedWorkout>();
var customQuery = query.OfType<CustomWorkout>()
    .Where(cw => cw.UserId == userId);

return predefinedQuery.Cast<WorkoutEntity>()
    .Concat(customQuery.Cast<WorkoutEntity>());
```

الصورة (22): تجميع الاستعلامات في استعلام واحد عن طريق الـ Entity Framework

Response body

```
{
  "Code": 500,
  "Message": "Expression of type 'System.Linq.IQueryable`1[Domain.Entities.AppEntities.PredefinedWorkout]' cannot be used for parameter of type 'System.Linq.IQueryable`1[Domain.Entities.AppEntities.CustomWorkout]' of method 'System.Linq.IQueryable`1[Domain.Entities.AppEntities.CustomWorkout] Concat[CustomWorkout](System.Linq.IQueryable`1[Domain.Entities.AppEntities.CustomWorkout], System.Collections.Generic.IEnumerable`1[Domain.Entities.AppEntities.CustomWorkout])' (Parameter 'arg0')",
  "Data": null
}
```



Download

الصورة (23): الخطأ الناتج عن استعمال الطريقة الموضحة في الصورة 17

تم حل هذه المشكلة بكتابة الاستعلام عن طريق تعابير لامدا Lambda Expression

```
query = query.Where(x => x is PredefinedWorkout || (x is CustomWorkout && ((CustomWorkout)x).UserId == userId));
return query;
```

الصورة (24): تجميع الاستعلامات عن طريق Lambda Expression

3.1.9 اختبار الضغط

```
scenarios: {  
  // Scenario 1: A user who creates, updates, and deletes workouts  
  power_user_workout_flow: [...],  
  
  // Scenario 2: A user who only browses data (read-only)  
  browsing_user_flow: [...],  
  
  // Scenario 3: An admin user performing admin tasks  
  admin_user_flow: [...],  
}
```

الصورة (25): السيناريوهات التي تم اعتمادها لإجراء اختبار الضغط

توضح الصورة 20 السيناريوهات الثلاث التي تم تطبيقها لإجراء اختبار الضغط:

- 1 **السيناريو الأول:** مجموعة من المستخدمين الافتراضيين الذين ينشئون قوائم تمارين ويضيفون تمارين إليها ويعدلون على تكرارات التمارين فيها ثم يحذفون قائمة التمارين.
- 2 **السيناريو الثاني:** مجموعة من المستخدمين الافتراضيين الذين يتصفحون قوائم التمارين جميعها ثم يتصفحون أحد هذه القوائم بشكل عشوائي.
- 3 **السيناريو الثالث:** مجموعة من المسؤولين الافتراضيين الذين يستعرضون جميع المستخدمين المسجلين، ثم يضيفون صنف جديد من التمارين ثم يعدلون عليه وفي الآخر يحذفونه.

```

// Stage Durations
const Round1 = '5s';
const Round2 = '20s';
const Round3 = '20s';
const Round4 = '10s';
const Round5 = '5s';

// Targets for Scenario 1:
const scenario1target1 = 10;
const scenario1target2 = 50;
const scenario1target3 = 50;
const scenario1target4 = 100;
const scenario1target5 = 0;

// Targets for Scenario 2:
const scenario2target1 = 10;
const scenario2target2 = 50;
const scenario2target3 = 50;
const scenario2target4 = 100;
const scenario2target5 = 0;

// Targets for Scenario 3:
const scenario3target1 = 10;
const scenario3target2 = 20;
const scenario3target3 = 20;
const scenario3target4 = 40;
const scenario3target5 = 0;

```

الصورة (26): المحددات للزمن وعدد المستخدمين الافتراضيين لكل فترة ضمن كل سيناريو

```
CUSTOM
successful_logins.....: 100.00% 2 out of 2

HTTP
http_req_duration.....: avg=344.41ms min=0s    med=128.22ms max=11.98s p(90)=944.34ms p(95)=1.31s
{ expected_response:true }.....: avg=370.64ms min=3.65ms med=145ms    max=11.98s p(90)=1.01s p(95)=1.35s
http_req_failed.....: 12.88% 591 out of 4585
http_reqs.....: 4585 70.432736/s

EXECUTION
iteration_duration.....: avg=2.24s min=1s    med=1.39s max=14.35s p(90)=5.01s p(95)=6.01s
iterations.....: 3018 46.361177/s
vus.....: 1 min=1 max=240
vus_max.....: 240 min=240 max=240

NETWORK

Output Error List Developer PowerShell
```

الصورة (27): نتائج اختبار الضغط

تظهر النتائج قدرة النظام على الاستجابة لطلبات المستخدمين والمسؤولين معا في ظل وجود ارتفاع مفاجئ أو الطلب المستمر، حيث كان متوسط زمن الاستجابة هو 344.41 ميلي ثانية وأعلى زمن هو 11.98 ثانية، تم تخديم 4585 طلب بمعدل 70 طلب في الثانية.

4.1.9 مناقشة النتائج

هذه القيم تم الحصول عليها باختبار النظام بوجود الرماز المصدري وقاعدة البيانات على الحاسب الشخصي، بينما عند استضافة هذه النظام مع قاعدة البيانات على مخدم مخصص، يتوقع استجابة أسرع للنظام. بعد التمعن في النتائج، تم التعديل على طريقة طلب الاستعلامات الخاصة بقوائم التمارين من قاعدة البيانات حيث تمثل المكون الرئيسي الذي يقوم عليه منطق العمل الخاص بهذا النظام، فبدلاً من طلب استعلام واحد يجمع قوائم التمارين (بنوعها الخاص بالمستخدم والمعد مسبقاً) مع كافة المعلومات ضمنها عن التمارين والصور المتعلقة بها، تم تقسيم الاستعلام عبر تعليمة `AsSplitQuery()`¹ التي توفرها الـ `Entity Framework`.

¹ تقوم بتقسيم الاستعلام الواحد إلى عدة استعلامات فرعية بدلاً من تنفيذ استعلام واحد كبير يحتوي على العديد من عمليات الانضمام (JOIN)، مما يقلل من التكرار الزائد في النتائج ويحسن الأداء في بعض الحالات.

```

CUSTOM
successful_logins.....: 100.00% 2 out of 2

HTTP
http_req_duration.....: avg=101.07ms min=0s    med=56.22ms max=2.49s p(90)=241.54ms p(95)=339.09ms
  { expected_response:true }.....: avg=113.23ms min=2.83ms med=68.15ms max=2.49s p(90)=261.45ms p(95)=362.22ms
http_req_failed.....: 12.45% 679 out of 5452
http_reqs.....: 5452 84.096139/s

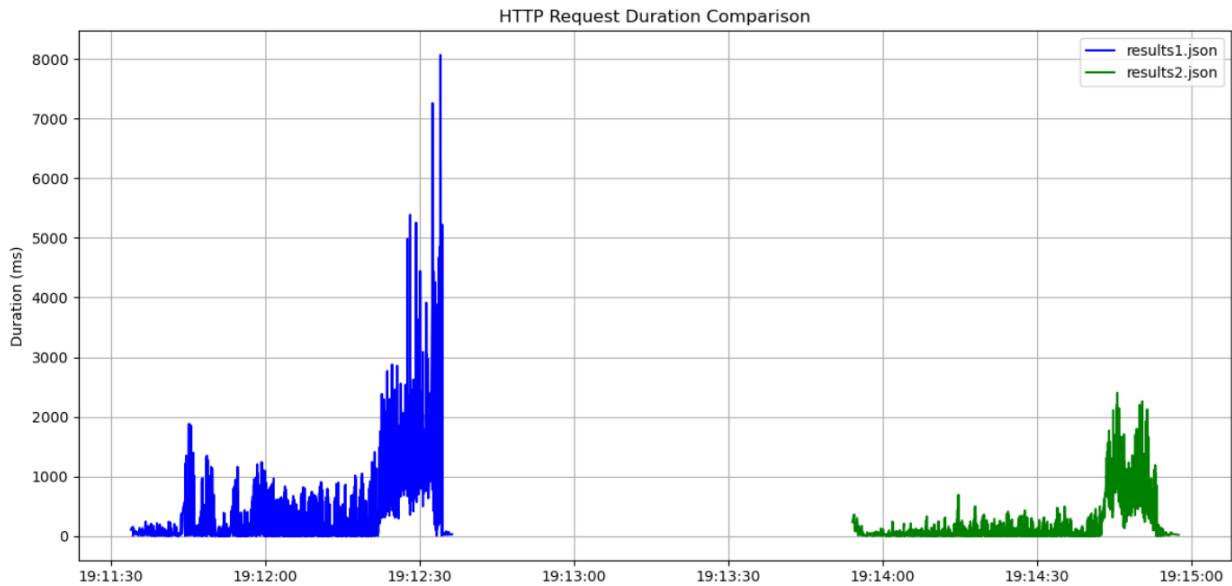
EXECUTION
iteration_duration.....: avg=1.85s min=1s    med=1.11s max=7.31s p(90)=4.11s p(95)=5.09s
iterations.....: 3602 55.560215/s
vus.....: 6 min=3 max=240
vus_max.....: 240 min=240 max=240

NETWORK

```

الصورة (28): نتائج اختبار الضغط بعد تقسيم الاستعلام الخاص بقوائم التمارين

تظهر الطريقة المتبعة تحسنا واضحا في النتائج، حيث أصبح متوسط الاستجابة 101.07 ميلي ثانية وأعلى زمن هو 2.49 ثانية، تم تخديم 5452 طلب بمعدل 84 طلب في الثانية.



الصورة (29): المقارنة بين الأزمنة لتخديم الطلبات قبل وبعد التعديل على طريقة استعلام قوائم التمارين حيث يظهر الخط الأزرق النتائج قبل التعديل

خاتمة وآفاق مستقبلية

المراجع

1. Adolfsen, K. A. (2021). Valuation of Oslo Klatresenter AS: a fundamental analysis of a Norwegian climbing gym company (Master's thesis).
2. Allous, F. A. B. I. O. (2024). Optimizing User Engagement in Fitness Apps: A Comparative Analysis of Features and User Models (Bachelor's thesis, University of Twente).
3. Dian, F. J., Vahidnia, R., & Rahmati, A. (2020). Wearables and the Internet of Things (IoT), applications, opportunities, and challenges: A Survey. *IEEE access*, 8, 69200-69211.
4. Gordon, M., Althoff, T., & Leskovec, J. (2019, May). Goal-setting and achievement in activity tracking apps: a case study of MyFitnessPal. In *The World Wide Web Conference* (pp. 571-582).
5. Griffiths, C., Harnack, L., & Pereira, M. A. (2018). Assessment of the accuracy of nutrient calculations of five popular nutrition tracking applications. *Public health nutrition*, 21(8), 1495-1502.
6. Kercher, V. M., Kercher, K., Bennion, T., Yates, B. A., Feito, Y., Alexander, C., ... & De la Cámara, M. Á. (2021). Fitness trends from around the globe. *ACSM's Health & Fitness Journal*, 25(1), 20-31.
7. Klonoff, D. C. (2019). Behavioral theory: the missing ingredient for digital health tools to change behavior and increase adherence. *Journal of diabetes science and technology*, 13(2), 276-281.
8. Lopes, J. M., Silva, L. F., Massano-Cardoso, I., & Galhardo, A. (2024). Running Towards a Better Brand Attitude: How Gamification in Nike Run Club Can Help?. *Journal of the Knowledge Economy*, 1-29.
9. Nazarenko, A. (2020). Updating the Contemporary Business Model for Fitness Market.
10. Or, C. K., Holden, R. J., & Valdez, R. S. (2022). Human factors engineering and user-centered design for mobile health technology: enhancing effectiveness, efficiency, and satisfaction. In *Human-Automation interaction: mobile computing* (pp. 97-118). Cham: Springer International Publishing.
11. Sagdiddinov, M. (2025). How Netflix, Peloton, And Zoom Redefined Crisis Leadership in Times of Chaos. *American Journal Of Social Sciences And Humanity Research*, 5(04), 179-187.
12. Sekhon, T. S. (2022). Marketing Plan.
13. Stancu, V., Frank, D. A., Lähteenmäki, L., & Grunert, K. G. (2022). Motivating consumers for health and fitness: The role of app features. *Journal of Consumer Behaviour*, 21(6), 1506-1521.
14. Tsai, T. H., Chang, Y. S., Chang, H. T., & Lin, Y. W. (2021). Running on a social exercise platform: Applying self-determination theory to increase motivation to participate in a sporting event. *Computers in Human Behavior*, 114, 106523.